Deep Learning Made Simple

From neurons to Transformers, LLMs and diffusion – concepts first, key equations only

Neural networks · Backpropagation · CNNs · Detection & Segmentation
RNN · LSTM · GRU · Attention · GANs & VAEs · NLP · Transformers
BERT · LLMs · Diffusion · Stable Diffusion & DALL-E · Video generation

Prepared by Rahul Kothuri

How to use this guide

This is the companion to *Machine Learning Made Simple*. It follows the course flow – neural networks, CNNs, detection and segmentation, RNNs and attention, generative models, NLP, Transformers, BERT, LLMs, diffusion and video generation – and explains each idea **in plain words first, then with a diagram, then with only the equations that matter**.

Small concepts are kept short. Model explanations are given space, with examples. The hands-on sessions (Keras MNIST, CNN image classification, the end-to-end CV project, LSTM forecasting, BERT fine-tuning, LLM APIs) are summarised in short grey "Hands-on in brief" boxes that list the steps and ideas, not code.

Box	What it holds
The idea in plain words	the concept without jargon
Real-life analogy	something familiar to anchor it
Key equations	the formulas worth knowing, with every symbol explained
Worked example / Where it is used	numbers, applications
Watch out	common mistakes and limits

Symbols. x = input, y = true answer, $\hat{y}$ = prediction, w = weight, b = bias, σ = sigmoid, η = learning rate, h_t = hidden state at time t, $\odot$ = element-wise multiplication, L = loss.

Most charts were generated from real computations (convolutions on a real photo, real noising for diffusion, real positional encodings, real training runs). Diagrams marked *illustrative* show the idea rather than a measured result.

Contents

How to use this guide **1**

1 Neural Networks & Deep Learning Fundamentals **3**
1.1 What is deep learning? . . . 3
1.2 The artificial neuron . . . 3
1.3 Multi-layer perceptron (MLP / ANN) . . . 4
1.4 Activation functions . . . 5
1.5 Loss functions . . . 5

2 ANN Training: Backpropagation & Optimisation **6**
2.1 The training loop . . . 6
2.2 Backpropagation . . . 6
2.3 Optimisers . . . 7
2.4 Making deep networks train well . . . 8
2.5 Key hyperparameters . . . 10

3 Convolutional Neural Networks (CNN) **11**
3.1 Why not a plain ANN for images? . . . 11
3.2 The convolution operation . . . 11
3.3 Pooling . . . 12
3.4 A complete CNN . . . 12

4 CNN Architectures: Classic to Modern **14**
4.1 LeNet-5 (1998) . . . 14
4.2 AlexNet (2012) . . . 14
4.3 VGG-16 / VGG-19 (2014) . . . 15
4.4 Advanced architectures . . . 16
4.5 Transfer learning . . . 18

5 Object Detection, Segmentation & CNN Visualisation **20**
5.1 Computer-vision tasks . . . 20
5.2 Object detection basics . . . 20
5.3 Two-stage detectors: the R-CNN family . . . 22
5.4 One-stage detectors . . . 22
5.5 Image segmentation . . . 23
5.6 CNN visualisation: what is the network looking at? . . . 25

6 Recurrent Neural Networks, LSTM & GRU **27**
6.1 Why sequence models? . . . 27
6.2 The simple RNN . . . 27
6.3 The vanishing gradient problem . . . 28
6.4 LSTM – Long Short-Term Memory (1997) . . . 29
6.5 GRU – Gated Recurrent Unit (2014) . . . 30
6.6 Bidirectional RNNs . . . 30
6.7 Comparison . . . 31

7 Sequence-to-Sequence Models & Attention **32**
7.1 Encoder–decoder (seq2seq) models . . . 32
7.2 The attention mechanism . . . 32

8 Generative Models: Autoencoders, GANs & VAEs **35**
8.1 Discriminative vs generative . . . 35
8.2 Autoencoders . . . 35
8.3 GANs – Generative Adversarial Networks (2014) . . . 35
8.4 Advanced GAN variants . . . 37
8.5 VAEs – Variational Autoencoders (2013) . . . 37
8.6 GANs, VAEs and modern GenAI . . . 38

9 NLP: Classical Text Processing & Word Embeddings **39**
9.1 What is NLP? . . . 39
9.2 Classical text processing . . . 39
9.3 Turning text into numbers . . . 40
9.4 Word embeddings . . . 41
9.5 Classical NLP pipelines . . . 42
9.6 From sequence models to Transformers . . . 42

10 Transformers **43**
10.1 Attention before Transformers – a quick recap . . . 43
10.2 Self-attention . . . 43
10.3 Multi-head attention . . . 44
10.4 Positional encoding . . . 45
10.5 The full Transformer architecture . . . 45
10.6 Why Transformers work so well . . . 46
10.7 Three families of Transformers . . . 46

11 BERT: Architecture, Pre-training & Variants **47**
11.1 What is BERT? . . . 47
11.2 Input representation . . . 47
11.3 Pre-training tasks . . . 47
11.4 Fine-tuning BERT for tasks . . . 48
11.5 BERT variants . . . 49
11.6 Making models smaller and faster . . . 49

12 Large Language Models (LLMs) **50**
12.1 What is an LLM? . . . 50
12.2 LLM applications . . . 51
12.3 The LLM landscape . . . 52
12.4 API integration . . . 52

13 Generative AI & Diffusion Models **53**
13.1 Introduction to generative AI . . . 53
13.2 Diffusion models: theory . . . 53
13.3 CLIP: connecting text and images . . . 55
13.4 Stable Diffusion . . . 55
13.5 DALL-E . . . 56
13.6 Image generation in practice . . . 56
13.7 Video generation models . . . 56

14 Cheat Sheet & Interview Revision **58**
14.1 Which architecture for which data? . . . 58
14.2 Formula sheet . . . 59
14.3 Interview quick revision . . . 59

Further Reading 61

CHAPTER 1

Neural Networks & Deep Learning Fundamentals

1.1 What is deep learning?

The idea in plain words

Deep learning is machine learning with **neural networks that have many layers**. Each layer turns its input into a slightly more useful representation. Early layers find simple patterns (edges, letters), later layers combine them into complex ones (faces, meanings). The key difference from classical ML: **the network learns the features itself** – you don't hand-craft them.

	Classical ML	Deep learning
Features	Designed by humans	Learned automatically
Data needed	Works with small data	Shines with large data
Best on	Tables (rows/columns)	Images, audio, text, video
Compute	CPU is fine	Usually needs GPUs
Interpretability	Often good	Harder ("black box")

Why now? Three things came together around 2012: **big data** (e.g. ImageNet), **GPUs** for fast matrix maths, and **better techniques** (ReLU, dropout, good initialisation, Adam).

1.2 The artificial neuron

The idea in plain words

A neuron multiplies each input by a weight (its importance), adds them up with a bias, and passes the result through an **activation function** that decides how strongly to "fire".

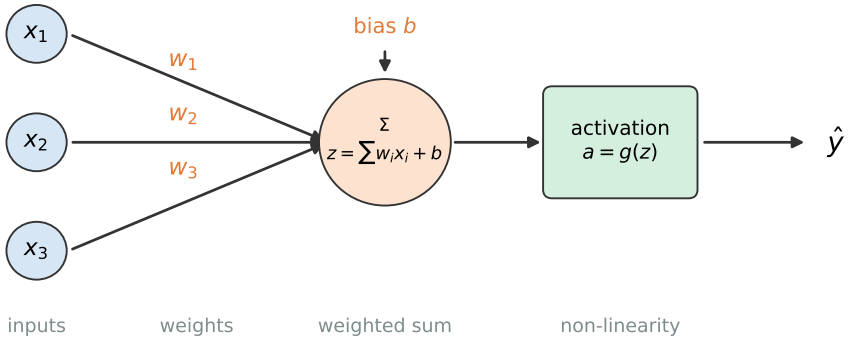


Figure 1.1: One artificial neuron.

Key equations

$$z = w_1x_1 + w_2x_2 + \cdots + w_nx_n + b = \mathbf{w}^\top \mathbf{x} + b, \qquad a = g(z)$$

w = weights, b = bias (shifts the threshold), g = activation function, a = the neuron's output.

Worked example

Inputs $x = (2, 3)$, weights $w = (0.5, -1)$, bias $b = 1$: $z = 0.5 \cdot 2 - 1 \cdot 3 + 1 = -1$. With ReLU, $a = \max(0, -1) = 0$ (the neuron stays silent). With sigmoid, $a = 1/(1 + e^1) \approx 0.27$.

1.2.1 The perceptron and its limit

The perceptron (Rosenblatt, 1958) is a single neuron with a step activation: output 1 if $z > 0$, else 0. It can only draw a **straight line**, so it cannot learn XOR. Stacking neurons into layers removes this limit.

1.3 Multi-layer perceptron (MLP / ANN)

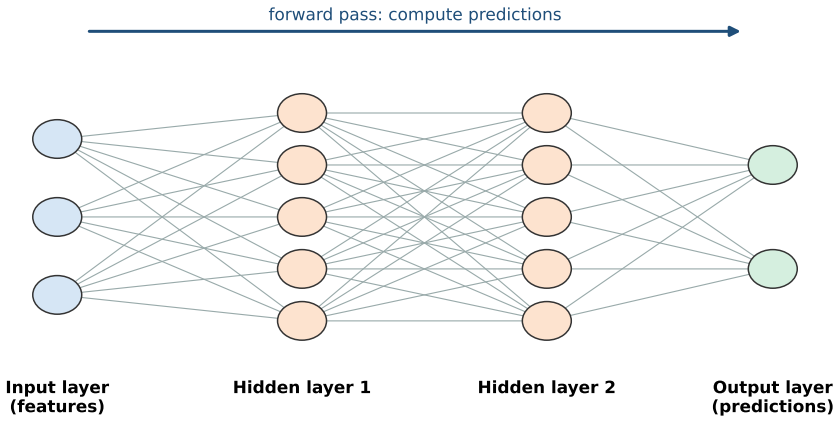


Figure 1.2: A fully-connected network: every neuron connects to every neuron in the next layer.

- **Input layer** – one node per feature (e.g. 784 pixels for a 28×28 image).
- **Hidden layers** – learn intermediate patterns. "Deep" = several hidden layers.
- **Output layer** – one node for regression; one per class (with softmax) for classification.

Key equations

Layer by layer: $\mathbf{a}^{(l)} = g(W^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)})$

Parameters of a dense layer = (inputs × outputs) + outputs (weights + biases).

Remember

Universal approximation theorem: a network with one hidden layer and enough neurons can approximate almost any continuous function. Deep networks do the same with far fewer neurons by reusing features layer after layer.

1.4 Activation functions

The idea in plain words

Without activations, stacking layers is pointless – many linear layers collapse into one linear layer. Activations add **non-linearity**, letting the network bend its decision boundaries.

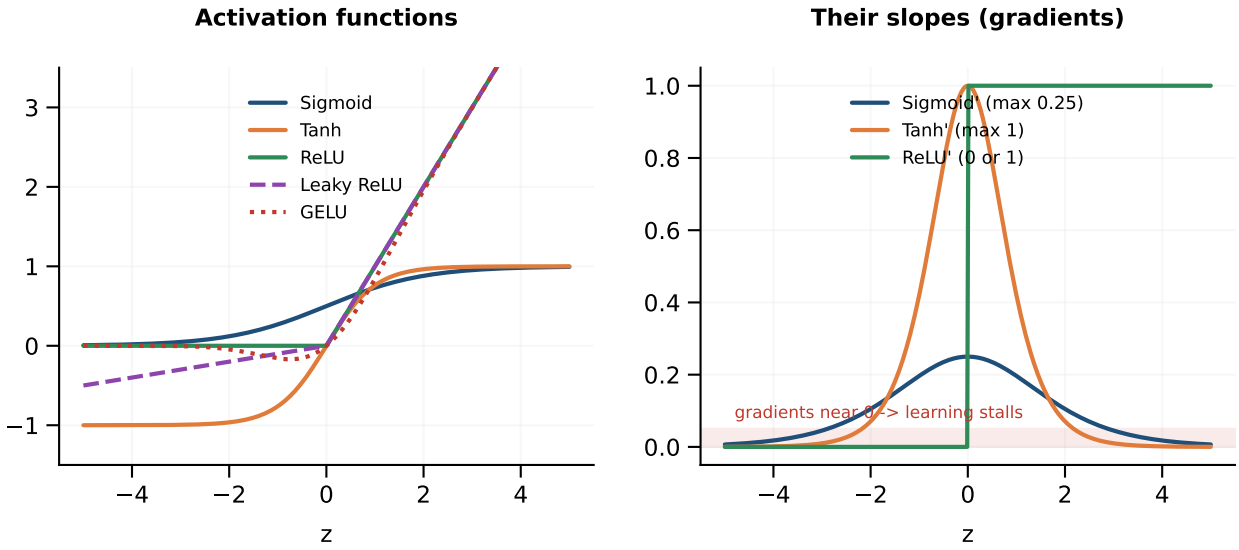


Figure 1.3: Left: common activations. Right: their slopes. Sigmoid's slope is at most 0.25, so gradients shrink layer after layer; ReLU's slope is 1 for positive inputs.

Activation	Formula	Use / note
Sigmoid	$\sigma(z) = \frac{1}{1+e^{-z}}$, range (0,1)	Binary output layer; vanishing gradient in hidden layers
Tanh	$\tanh(z)$, range $(-1, 1)$	Zero-centred; RNNs/LSTMs
ReLU	$\max(0, z)$	Default for hidden layers; fast; "dying ReLU" if stuck at 0
Leaky ReLU	$\max(0.01z, z)$	Fixes dying ReLU
GELU	$z \cdot \Phi(z)$ (smooth ReLU)	Transformers (BERT, GPT)
Softmax	$\frac{e^{z_k}}{\sum_j e^{z_j}}$	Multi-class output; probabilities sum to 1

1.5 Loss functions

Task	Loss	Output layer
Regression	MSE $= \frac{1}{n}\sum(y-\hat{y})^2$ (or MAE, Huber)	1 linear neuron
Binary classification	Binary cross-entropy $-[y \log \hat{p} + (1-y)\log(1-\hat{p})]$	1 sigmoid neuron
Multi-class	Categorical cross-entropy $-\sum_k y_k \log \hat{p}_k$	K softmax neurons

In Keras: `categorical_crossentropy` expects one-hot labels; `sparse_categorical_crossentropy` expects integer labels (0–9).

CHAPTER 2

ANN Training: Backpropagation & Optimisation

2.1 The training loop

The idea in plain words

Training repeats four steps thousands of times: (1) **forward pass** – make predictions; (2) **loss** – measure the error; (3) **backward pass (backpropagation)** – work out how much each weight contributed to the error; (4) **update** – nudge every weight to reduce the error.

- **Batch** – a small group of examples (e.g. 32) processed together.
- **Iteration (step)** – one weight update (one batch).
- **Epoch** – one full pass over the training data. 60,000 images with batch 32 $\Rightarrow$ 1,875 steps per epoch.

2.2 Backpropagation

The idea in plain words

Backpropagation is the **chain rule** from calculus applied backwards through the network. Starting at the loss, it passes "blame" back layer by layer, telling each weight: *if you increase a little, the loss changes by this much.*

Real-life analogy

A football team loses a match. The coach traces the goal back: the defender's slip, the midfielder's lost ball, the striker's missed chance – each player gets feedback proportional to their part in the mistake.

Key equations

Chain rule: $\dfrac{\partial L}{\partial w} = \dfrac{\partial L}{\partial a} \cdot \dfrac{\partial a}{\partial z} \cdot \dfrac{\partial z}{\partial w}$

Weight update (gradient descent): $w \leftarrow w - \eta \dfrac{\partial L}{\partial w}$

Worked example

One neuron, $x = 2$, $w = 0.5$, $b = 0.1$, sigmoid activation, true $y = 1$, loss $L = (a - y)^2$.

1. Forward: $z = 0.5 \cdot 2 + 0.1 = 1.1$, $a = \sigma(1.1) = 0.750$, $L = (0.750 - 1)^2 = 0.0624$.
2. Backward: $\frac{\partial L}{\partial a} = 2(a - y) = -0.50$; $\frac{\partial a}{\partial z} = a(1 - a) = 0.1875$; $\frac{\partial z}{\partial w} = x = 2$.
3. Multiply: $\frac{\partial L}{\partial w} = -0.50 \times 0.1875 \times 2 \approx -0.187$.
4. Update with $\eta = 0.5$: $w = 0.5 - 0.5 \times (-0.187) = 0.594$. The weight grows, z grows, a moves closer to 1 – the loss falls.

In a deep network the same idea repeats for every layer; the gradient for an early layer is a **product** of many terms – the source of vanishing and exploding gradients.

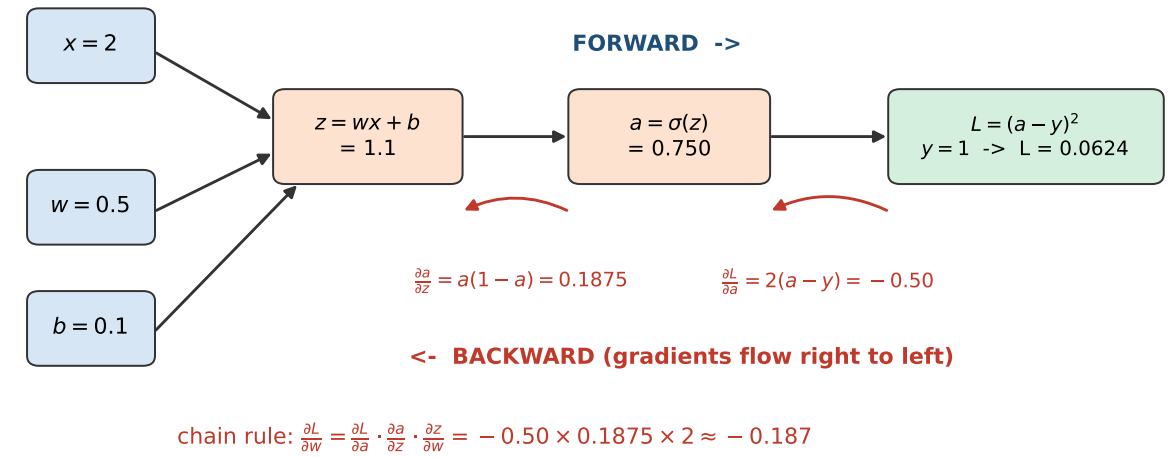


Figure 2.1: Forward values (blue arrows) and backward gradients (red) for one neuron.

2.3 Optimisers

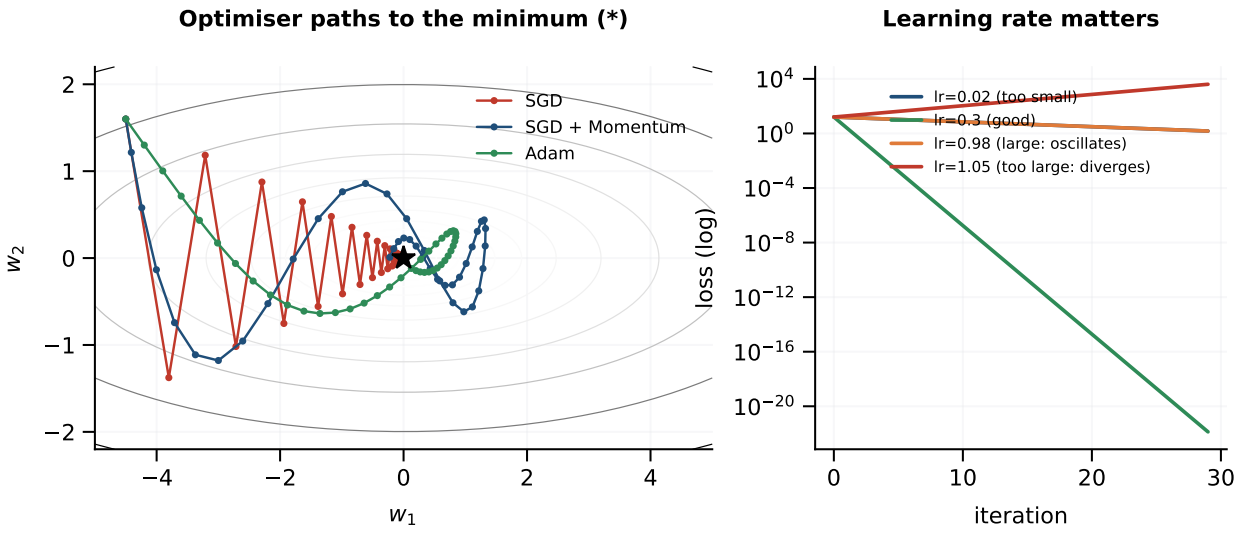


Figure 2.2: Left: plain SGD zig-zags; momentum and Adam reach the minimum more smoothly. Right: the learning rate decides whether training crawls, converges or blows up.

Optimiser	Idea in one line
SGD (mini-batch)	$w \leftarrow w - \eta g$ using the gradient g of one batch
Momentum	Keep a running velocity: $v \leftarrow \beta v + \eta g$, $w \leftarrow w - v$ (like a rolling ball)
RMSProp	Divide each step by the recent size of that weight's gradients (per-weight step size)
Adam	Momentum + RMSProp, with bias correction; default $\eta = 0.001$, $\beta_1 = 0.9$, $\beta_2 = 0.999$. The usual first choice
AdamW	Adam with decoupled weight decay; standard for Transformers

Learning-rate schedules (step decay, cosine decay, warm-up, `ReduceLROnPlateau`) start with bigger steps and finish with small, careful ones.

2.4 Making deep networks train well

2.4.1 Weight initialisation

All-zero weights make every neuron identical; too-large weights explode. Good defaults keep the signal's variance stable through the layers:

Key equations

Xavier/Glorot (tanh, sigmoid): $\mathrm{Var}(w) = \dfrac{2}{n_{\text{in}} + n_{\text{out}}}$ **He** (ReLU): $\mathrm{Var}(w) = \dfrac{2}{n_{\text{in}}}$

2.4.2 Batch normalisation

The idea in plain words

Normalise each layer's inputs over the batch to mean 0 and standard deviation 1, then let the network re-scale them with two learnable numbers. Training becomes faster and more stable, and higher learning rates work.

Key equations

$$\hat{x} = \frac{x - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, \qquad y = \gamma \hat{x} + \beta \qquad (\mu_B, \sigma_B = \text{batch mean and std; } \gamma, \beta \text{ learned})$$

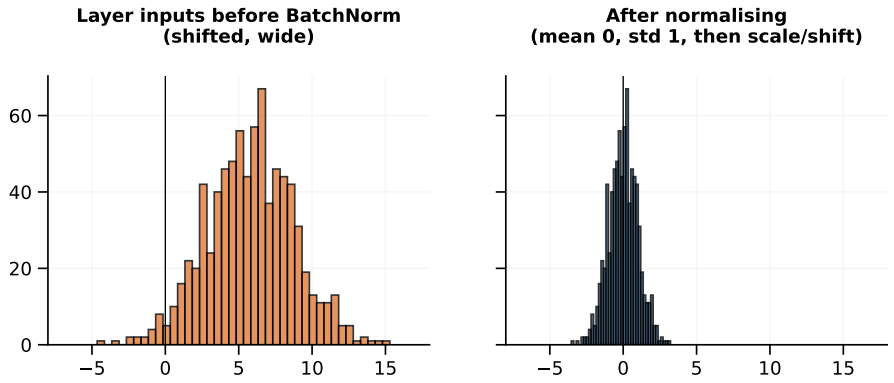


Figure 2.3: Batch normalisation re-centres and re-scales activations.

Layer normalisation does the same per example across features – used in Transformers and RNNs where batch statistics are awkward.

2.4.3 Regularisation: fighting overfitting

- **Dropout** – during training, randomly switch off a fraction p of neurons (e.g. 0.2–0.5) so the network can't rely on any single one. At test time all neurons are used (Keras rescales automatically).
- **L2 weight decay** – add $\lambda \sum w^2$ to the loss.
- **Early stopping** – stop when validation loss stops improving (`patience` epochs) and restore the best weights.
- **Data augmentation** – flips, crops, rotations, noise: more variety for free.

2.4.4 Vanishing and exploding gradients

Backprop multiplies many derivatives together. If they are mostly < 1 (sigmoid's slope is at most 0.25), the product **vanishes** and early layers stop learning. If mostly > 1, it **explodes** and training

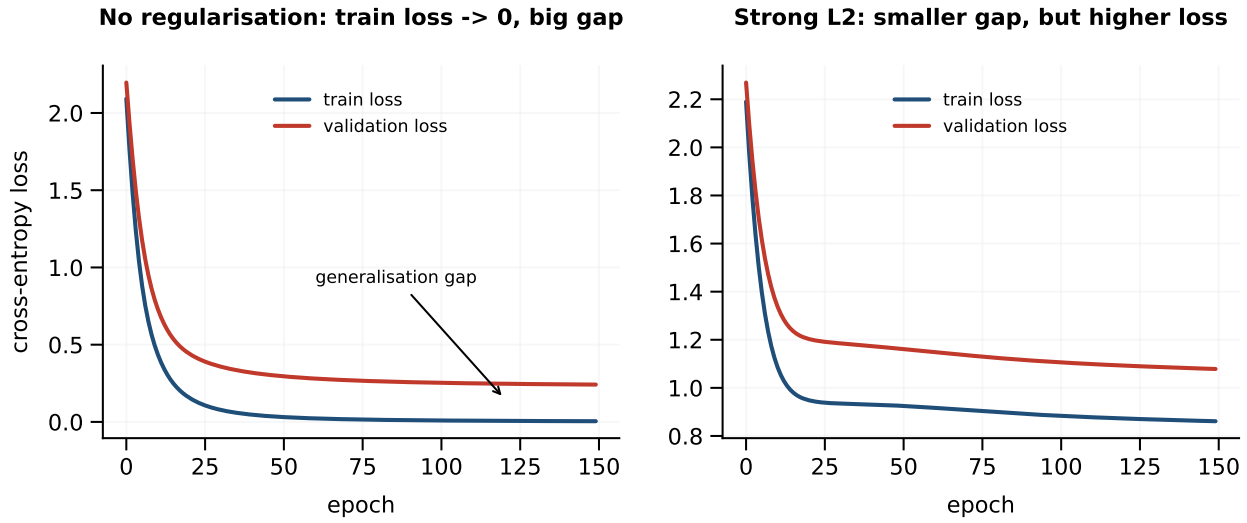


Figure 2.4: Real training run (small network, 150 digit images). Left: training loss goes to almost zero while validation loss stays higher – the gap is overfitting. Right: strong L2 regularisation narrows the gap, but too much raises both losses.

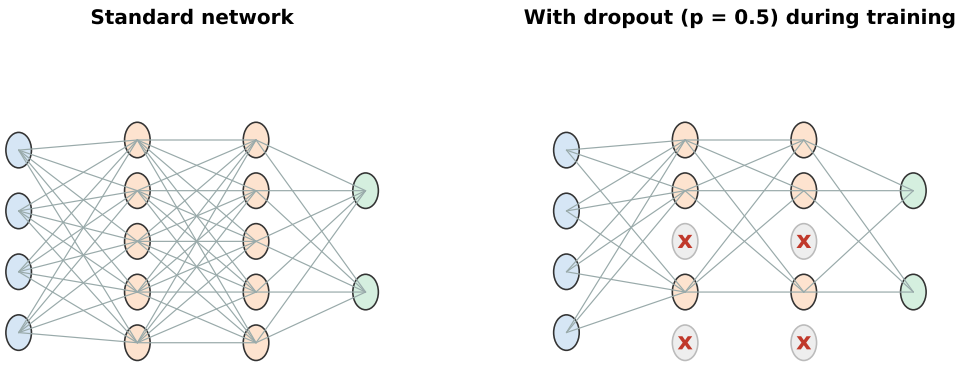


Figure 2.5: Dropout removes random neurons in each training step.

diverges (loss becomes NaN). Fixes: ReLU, He/Xavier init, batch norm, residual connections, gradient clipping. (This returns in RNNs, Chapter 6.)

2.5 Key hyperparameters

Hyperparameter	Effect	Typical start
Learning rate	Most important; too high diverges, too low is slow	1e-3 (Adam)
Batch size	Larger = smoother, faster per epoch, more memory	32–256
Epochs	Too many overfits	Use early stopping
Layers / neurons	Capacity	Start small, grow if underfitting
Dropout rate	Regularisation strength	0.2–0.5

Hands-on in brief

ANN with Keras on MNIST (70,000 handwritten digits, 28×28 greyscale, 10 classes).

1. Load data, scale pixels to $[0, 1]$ (divide by 255).
2. Build `Sequential`: `Flatten` → `Dense(128, relu)` → `Dropout(0.2)` → `Dense(10, softmax)`.
3. `compile(optimizer="adam", loss="sparse_categorical_crossentropy", metrics=["accuracy"])`.
4. `fit(..., epochs=5-10, validation_split=0.1)`, then `evaluate` on the test set.

A simple MLP like this typically reaches about 97–98% test accuracy; a CNN usually reaches about 99%.

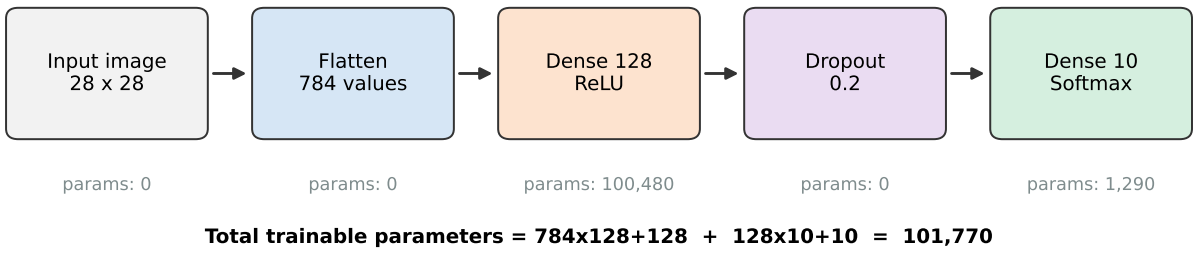


Figure 2.6: The MNIST MLP and where its 101,770 parameters come from.

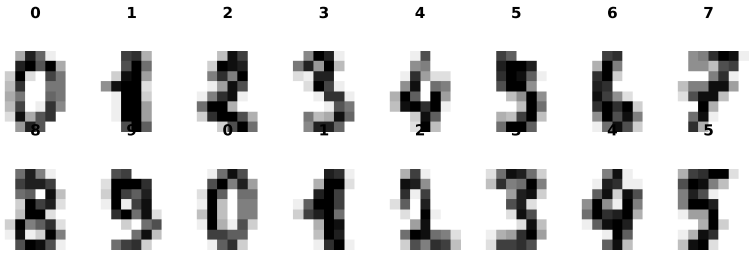


Figure 2.7: Handwritten digit samples (scikit-learn's 8×8 digits, a small cousin of MNIST).

CHAPTER 3

Convolutional Neural Networks (CNN)

3.1 Why not a plain ANN for images?

A 224×224 colour image has $224 \times 224 \times 3 = 150{,}528$ inputs. One dense layer of 1,000 neurons would already need about 150 million weights – and it would treat a cat in the top-left corner as completely different from the same cat in the bottom-right. CNNs fix both problems with three ideas:

1. **Local connectivity** – each neuron looks at a small patch (e.g. 3×3), because nearby pixels are related.
2. **Weight sharing** – the same small filter slides over the whole image, so a detector learned in one place works everywhere (*translation invariance*) with very few parameters.
3. **Hierarchy** – stacking layers builds edges → textures → parts → objects.

3.2 The convolution operation

The idea in plain words

A **filter (kernel)** is a small grid of weights. Slide it over the image; at each position multiply overlapping numbers and add them up. The result is a **feature map** that is bright wherever the image matches the filter's pattern.

Real-life analogy

Running a magnifying glass that only lights up when it sees a vertical edge across a page – the trail of lights is the feature map.

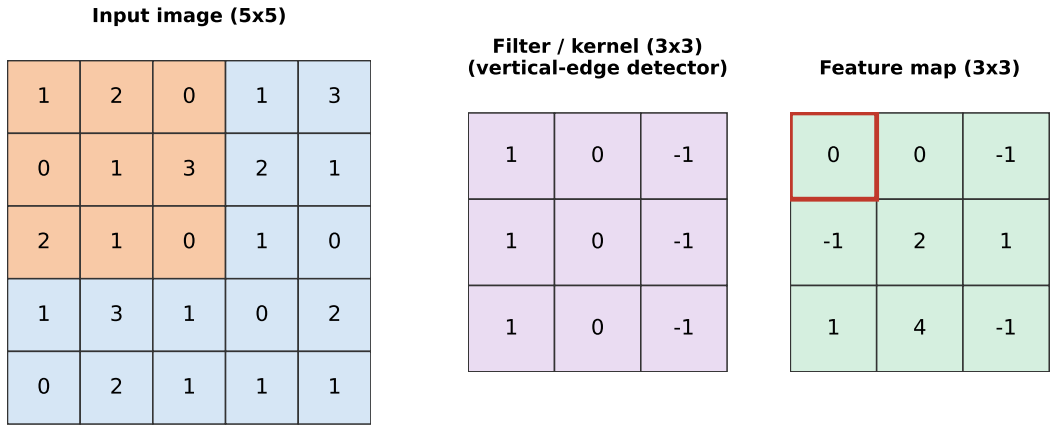


Figure 3.1: A 3×3 vertical-edge filter sliding over a 5×5 input gives a 3×3 feature map.

Key equations

Output size: $O = \left\lfloor \dfrac{W - K + 2P}{S} \right\rfloor + 1$ W = input size, K = kernel, P = padding, S = stride

Parameters of a conv layer: $(K \times K \times C_{\text{in}} + 1) \times C_{\text{out}}$

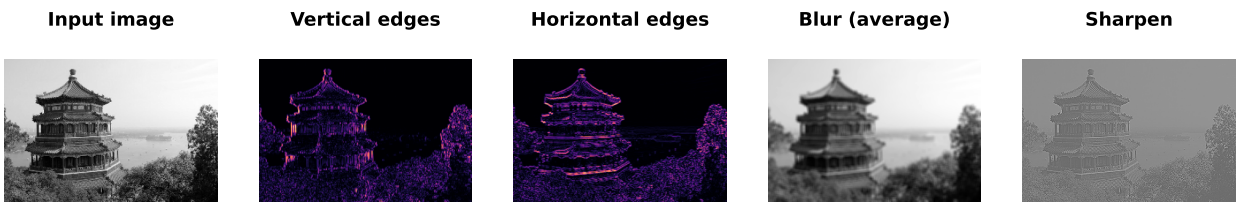


Figure 3.2: Real convolutions on a photo: different filters pull out different features. A CNN *learns* its filters instead of using hand-made ones.

Worked example

32×32 input, 5×5 kernel, no padding, stride 1: $O = (32 - 5 + 0)/1 + 1 = 28$.
With `padding="same"` ($P = 2$): $O = 32$ (size kept). With stride 2: $O = \lfloor (32 - 5)/2 \rfloor + 1 = 14$.
Parameters for 64 filters of 3×3 on an RGB image: $(3 \cdot 3 \cdot 3 + 1) \times 64 = 1{,}792$ – tiny compared with a dense layer.

- **Stride** – step size of the slide; stride 2 halves the output size.
- **Padding** – add a border of zeros; "valid" = none (shrinks), "same" = keeps size.
- **Channels** – a colour image has 3; each filter spans all input channels and produces one output channel. 64 filters ⇒ 64 feature maps.
- **1×1 convolution** – mixes channels without looking at neighbours; a cheap way to shrink or grow the number of channels.

3.3 Pooling

The idea in plain words

Pooling shrinks feature maps by summarising small regions, keeping the important signal while cutting computation and adding tolerance to small shifts.

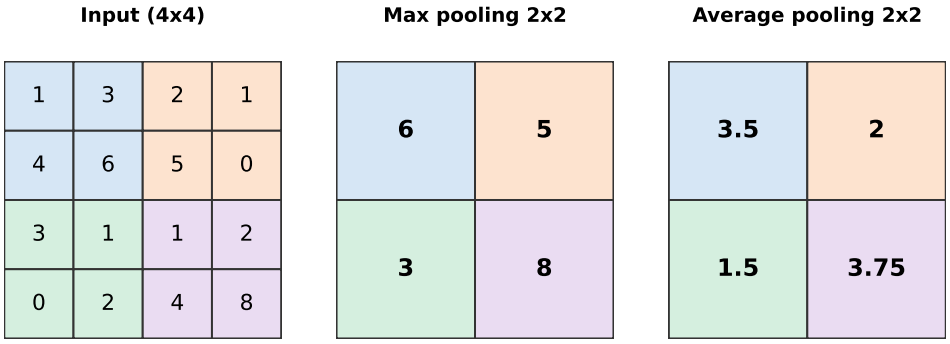


Figure 3.3: 2×2 pooling with stride 2 halves width and height. Max pooling keeps the strongest response; average pooling keeps the mean.

Pooling has **no learnable parameters**. **Global average pooling** averages each whole feature map to one number – modern networks use it instead of big dense layers.

3.4 A complete CNN

Remember

Receptive field = the patch of the original image a neuron "sees". It grows with depth, which is why deep layers can recognise whole objects.

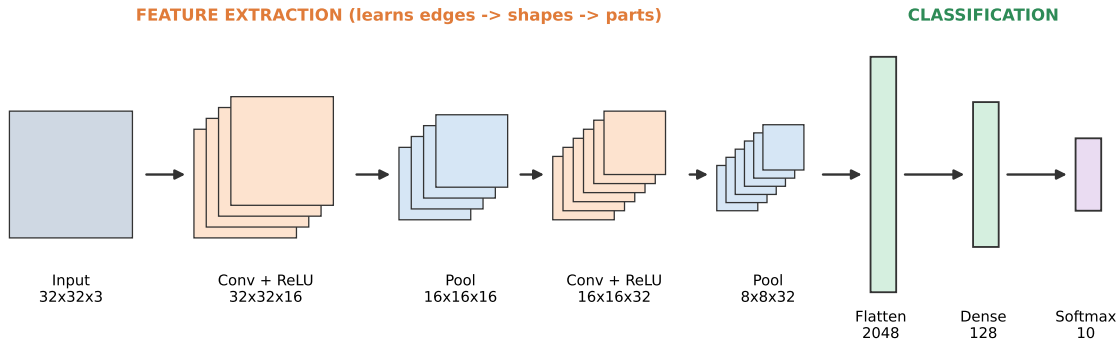


Figure 3.4: Typical CNN: [Conv → ReLU → Pool] blocks extract features; Flatten + Dense layers classify. Spatial size shrinks while depth (channels) grows.

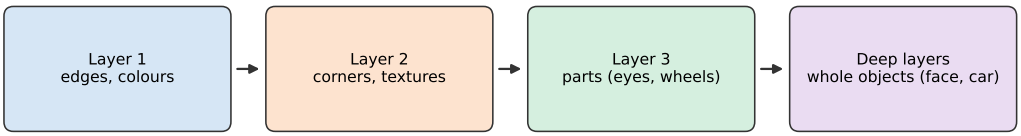


Figure 3.5: What successive layers tend to learn.

Hands-on in brief

Image classification with a CNN (e.g. CIFAR-10: 60,000 32×32 colour images, 10 classes).

1. Normalise pixels; optionally add augmentation layers (`RandomFlip`, `RandomRotation`, `RandomZoom`).
2. Stack 2–3 blocks of `Conv2D(32/64/128, 3, relu, padding="same")` + `MaxPooling2D`; add `BatchNormalization` and `Dropout`.
3. `Flatten` or `GlobalAveragePooling2D` → `Dense` → `Dense(10, softmax)`.
4. Train with Adam + early stopping; evaluate with accuracy and a confusion matrix to see which classes get mixed up (e.g. cat vs dog).

CHAPTER 4

CNN Architectures: Classic to Modern

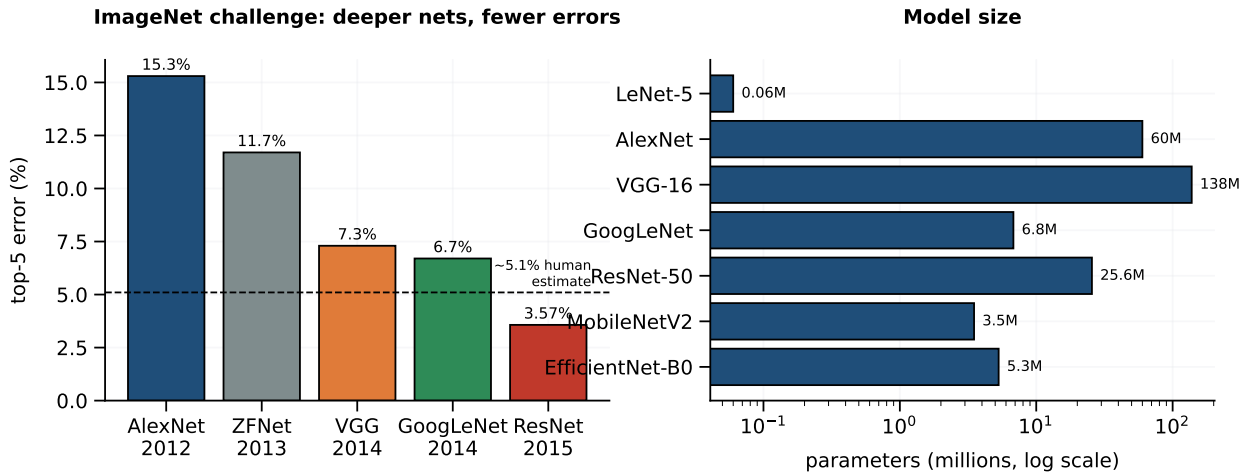


Figure 4.1: Left: ImageNet (ILSVRC) top-5 error of winning-era architectures. Right: model size – modern networks get better accuracy with far fewer parameters.

4.1 LeNet-5 (1998)

The idea in plain words

The first famous CNN, built by Yann LeCun to read handwritten digits on bank cheques. It established the pattern still used today: **conv → pool → conv → pool → fully connected**.

- Input 32×32 greyscale → Conv 6@5×5 (28×28) → avg-pool (14×14) → Conv 16@5×5 (10×10) → avg-pool (5×5) → FC 120 → FC 84 → 10 outputs.
- About **60 thousand parameters**; tanh/sigmoid activations.

4.2 AlexNet (2012)

The idea in plain words

The network that started the deep-learning boom: it won ImageNet 2012 with about 15.3% top-5 error, far ahead of the runner-up (about 26%). Same basic idea as LeNet, but much bigger and trained on GPUs.

- 8 learned layers: 5 conv + 3 fully connected; about **60 million parameters**.
- First layer: 96 filters of 11×11 with stride 4 on a 224×224×3 image.
- Key innovations: **ReLU** (much faster training than tanh), **dropout** (0.5) in the FC layers, heavy **data augmentation**, overlapping max-pooling, training on two GPUs.

4.3 VGG-16 / VGG-19 (2014)

The idea in plain words

VGG asked: what if we keep the design dead simple – **only 3×3 convolutions**, stacked deep, with 2×2 max-pooling between blocks? Uniform and easy to understand, it became a favourite backbone for transfer learning.

- VGG-16 = 13 conv + 3 FC layers (VGG-19 = 16 + 3). Channels double after each pool: 64 → 128 → 256 → 512 → 512.
- About **138 million parameters** – most of them in the first FC layer ($7 \cdot 7 \cdot 512 \times 4096 \approx 103$M).
- Second place in the ImageNet 2014 classification task (about 7.3% top-5 error).

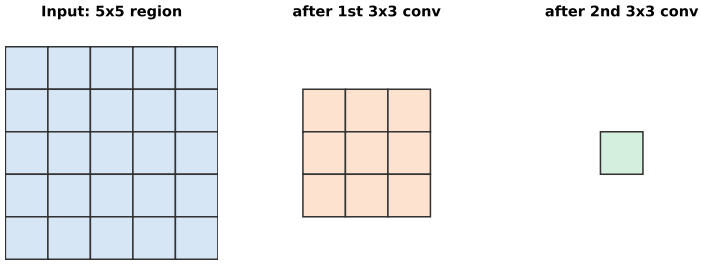


Figure 4.2: Why stacks of small filters: two 3×3 layers cover the same 5×5 area as one 5×5 layer, with fewer weights and an extra non-linearity.

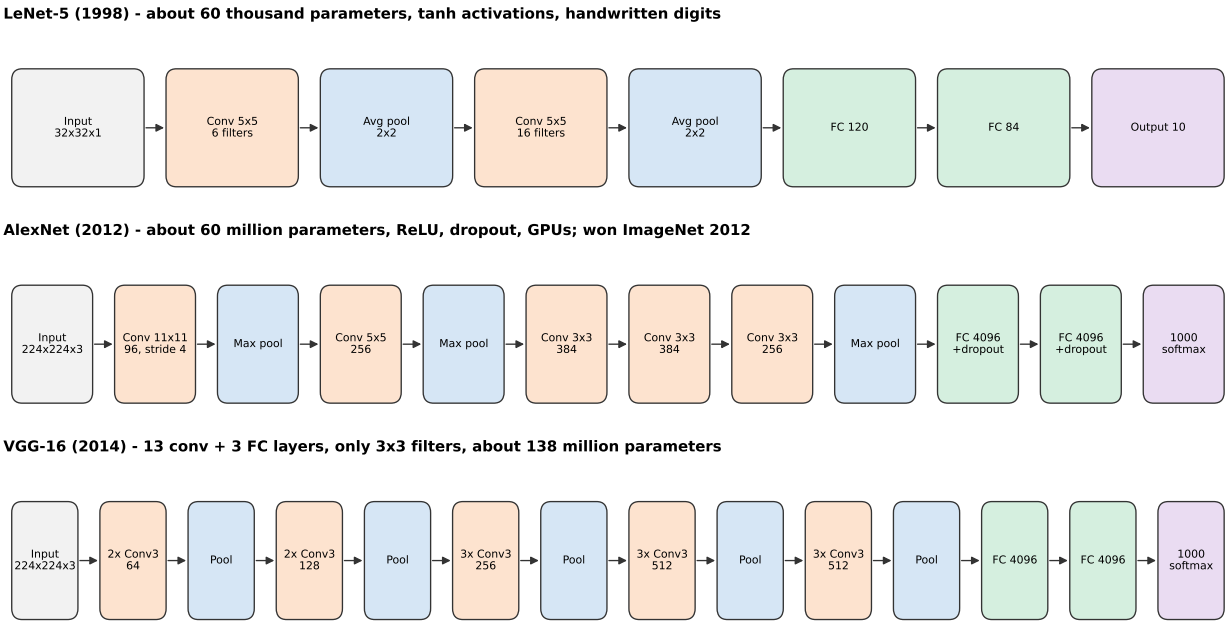


Figure 4.3: LeNet-5, AlexNet and VGG-16 at a glance (orange = conv, blue = pooling, green = fully connected).

Model	Year	Params	Key idea	Limitation
LeNet-5	1998	60K	Conv + pool pattern	Tiny images only
AlexNet	2012	60M	ReLU, dropout, GPUs, augmentation	Large irregular kernels
VGG-16	2014	138M	Only 3×3 convs, deep and uniform	Heavy, slow, huge FC layers
GoogLeNet	2014	≈7M	Inception modules, 1×1 bottlenecks	Complex design
ResNet-50	2015	25.6M	Skip connections, 50–152 layers	–
DenseNet	2017	8M (121)	Each layer gets all earlier maps	Memory-hungry
MobileNetV2	2018	3.5M	Depthwise separable convs	Slightly lower accuracy
EfficientNet-B0	2019	5.3M	Compound scaling	–

4.4 Advanced architectures

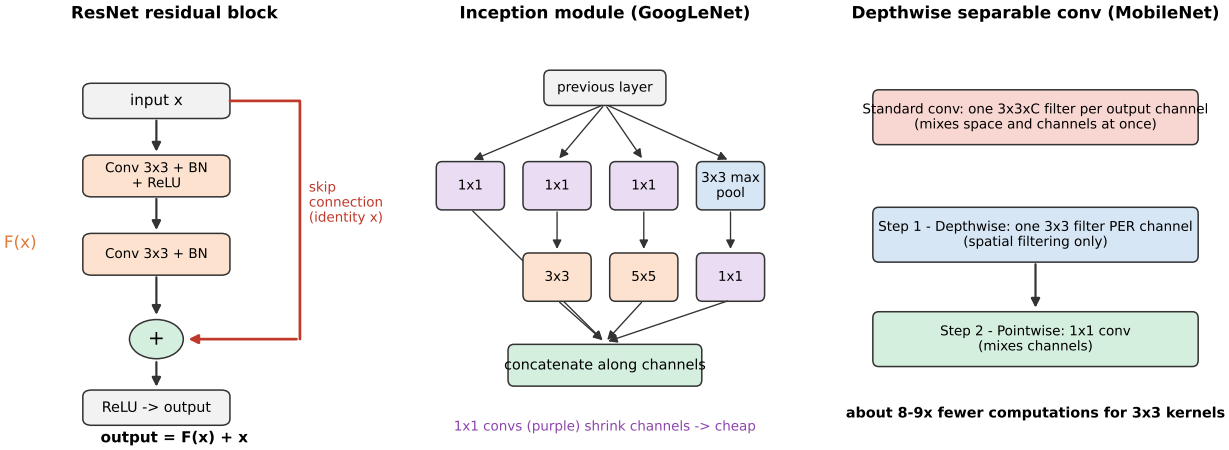


Figure 4.4: The building blocks behind modern CNNs.

4.4.1 GoogLeNet / Inception (2014)

The idea in plain words

Instead of choosing one filter size, an **Inception module** runs 1×1, 3×3, 5×5 convolutions and pooling *in parallel* and concatenates the results – the network decides which scale matters. 1×1 convolutions first shrink the channels so this stays cheap.

22 layers deep yet only about 7M parameters; won ImageNet 2014 (about 6.7% top-5 error). Uses global average pooling instead of large FC layers.

4.4.2 ResNet (2015) – residual networks

The idea in plain words

Very deep plain networks got *worse*, even on training data – they were hard to optimise. ResNet adds a **shortcut** that skips over a few layers and adds the input back. The layers now only need to learn the *difference* (residual) from the input. If a block isn't useful, it can simply learn to pass the input through.

Key equations

$$\text{output} = F(x) + x$$

The "$+x$" gives gradients a direct highway backwards, so networks with 50, 101 or even 152 layers train well.

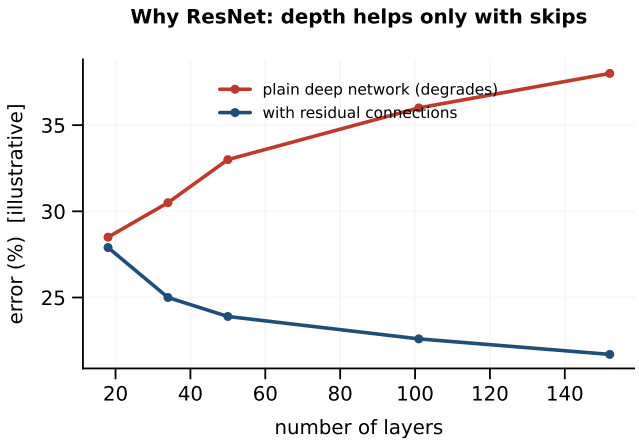


Figure 4.5: Illustrative: without skips, adding layers raises error; with residual connections deeper is better.

ResNet-152 won ImageNet 2015 with 3.57% top-5 error (ensemble), below the commonly quoted human estimate of about 5%. ResNet-50 (about 25.6M params) is still a standard backbone. ResNet-50 and deeper use a **bottleneck block** ($1\times1 \to 3\times3 \to 1\times1$).

4.4.3 DenseNet (2017)

Each layer receives the feature maps of *all* previous layers (concatenated). Strong feature reuse, fewer parameters, good gradient flow.

4.4.4 MobileNet (2017+)

The idea in plain words

Built for phones. It splits a normal convolution into a **depthwise** step (one filter per channel, spatial only) and a **pointwise** 1×1 step (mix channels).

Key equations

Cost ratio vs standard conv $\approx \frac{1}{N} + \frac{1}{K^2}$ (N output channels, K kernel size) $\Rightarrow$ about 8–9$\times$ cheaper for 3×3.

4.4.5 EfficientNet (2019)

The idea in plain words

Scale depth, width and input resolution *together* in a fixed ratio ("compound scaling") instead of just adding layers. B0 is small; B1–B7 are scaled-up versions with better accuracy per parameter.

4.4.6 Vision Transformers (ViT, 2020) – a note

Cut the image into 16×16 patches, treat each patch as a "word", and feed them to a Transformer (Chapter 10). With very large training data they match or beat CNNs.

4.5 Transfer learning

The idea in plain words

Don't start from zero. A network trained on ImageNet (1.2 million images, 1,000 classes) has already learned general visual features – edges, textures, shapes. Reuse that knowledge and only teach it your specific classes. You need much less data and training time.

Real-life analogy

Someone who already plays the guitar learns the ukulele far faster than a complete beginner.

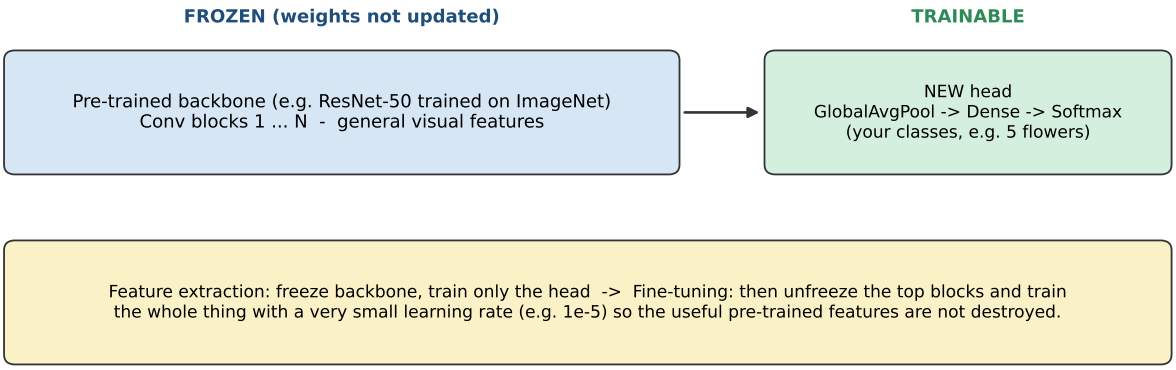


Figure 4.6: Feature extraction (freeze the backbone, train a new head), then optional fine-tuning.

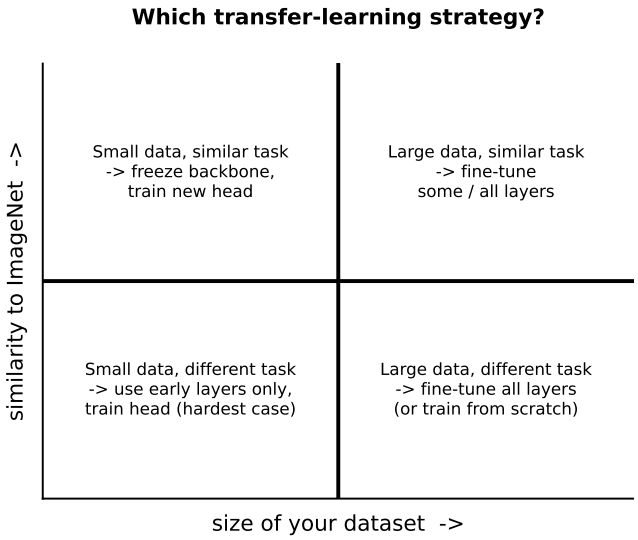


Figure 4.7: Choosing a strategy by data size and similarity to the original data.

1. Load a pre-trained model without its top: e.g. `ResNet50(weights="imagenet", include_top=False)`.
2. Freeze it (`trainable=False`); add `GlobalAveragePooling2D` + `Dense` + softmax for your classes.
3. Train the head. Use the model's own preprocessing function on your images.
4. **Fine-tune**: unfreeze the top few blocks and train again with a very low learning rate (e.g. 10^{-5}).

Watch out

Fine-tuning with a high learning rate destroys the pre-trained features ("catastrophic forgetting"). Keep BatchNorm layers in inference mode when fine-tuning in Keras.

CHAPTER 5

Object Detection, Segmentation & CNN Visualisation

5.1 Computer-vision tasks

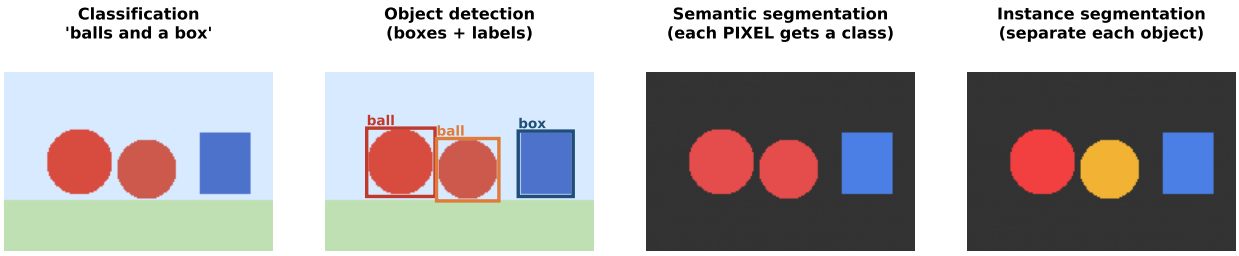


Figure 5.1: From "what is in the image" to "exactly which pixels belong to which object".

Task	Output	Example models
Classification	One label per image	ResNet, EfficientNet
Object detection	Box + label + score per object	Faster R-CNN, YOLO, SSD
Semantic segmentation	A class for every pixel	FCN, U-Net, DeepLab
Instance segmentation	A separate mask per object	Mask R-CNN
Panoptic segmentation	Semantic + instance together	Panoptic FPN, Mask2Former

5.2 Object detection basics

A box is described by 4 numbers, e.g. (x, y, w, h) = centre, width, height. The model must solve two problems at once: **where** (box regression) and **what** (classification).

5.2.1 IoU – Intersection over Union

Key equations

$$\text{IoU} = \frac{\text{area of overlap}}{\text{area of union}}$$

0 = no overlap, 1 = perfect. A detection usually counts as correct if IoU ≥ 0.5.

5.2.2 Non-Maximum Suppression (NMS)

Detectors produce many overlapping boxes for the same object. NMS: (1) take the box with the highest score; (2) delete other boxes that overlap it with IoU above a threshold (e.g. 0.5); (3) repeat with the remaining boxes.

5.2.3 Anchor boxes

Pre-defined boxes of several sizes and shapes placed at every location. The network predicts small *adjustments* to the anchors plus an "object or not" score – easier than predicting boxes from scratch.

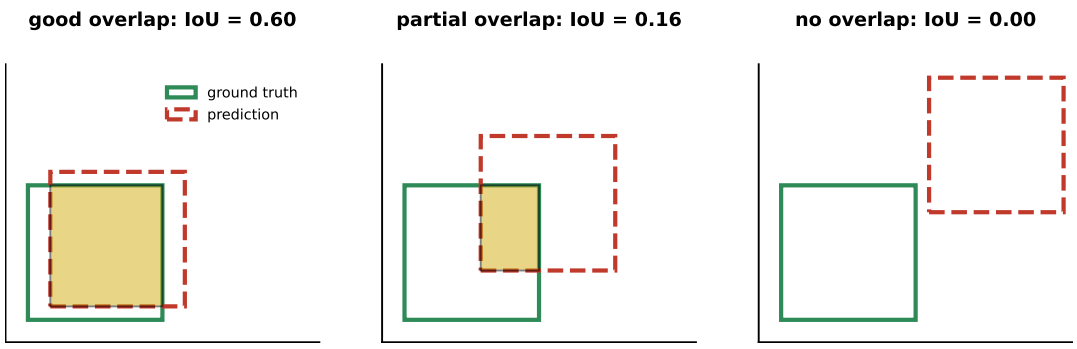


Figure 5.2: IoU for three predicted boxes against the same ground truth.

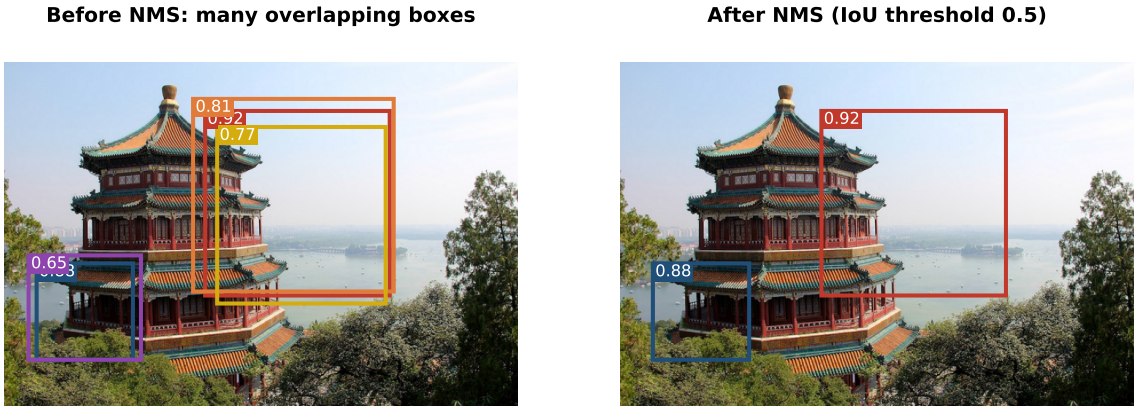


Figure 5.3: NMS keeps one confident box per object (illustrative scores).

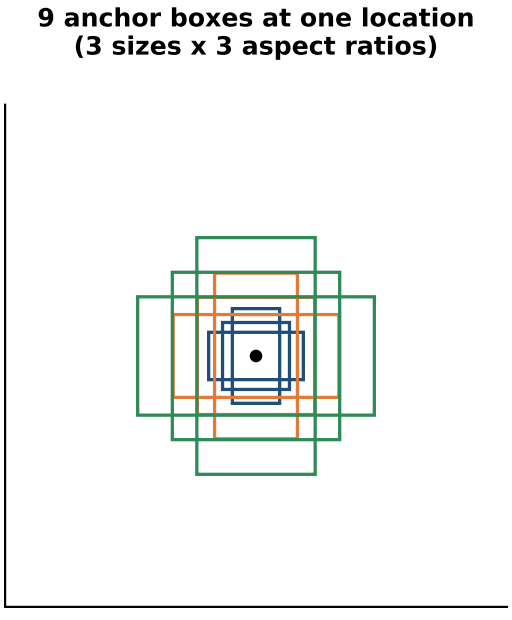


Figure 5.4: Anchors at one position.

5.2.4 Measuring detectors: mAP

For each class, rank detections by score and draw a precision–recall curve; **AP** = area under it. **mAP** = mean of AP over classes. Pascal VOC uses IoU 0.5 (mAP@0.5); COCO averages over IoU

0.5 to 0.95 (mAP@[.5:.95]) – stricter.

5.3 Two-stage detectors: the R-CNN family

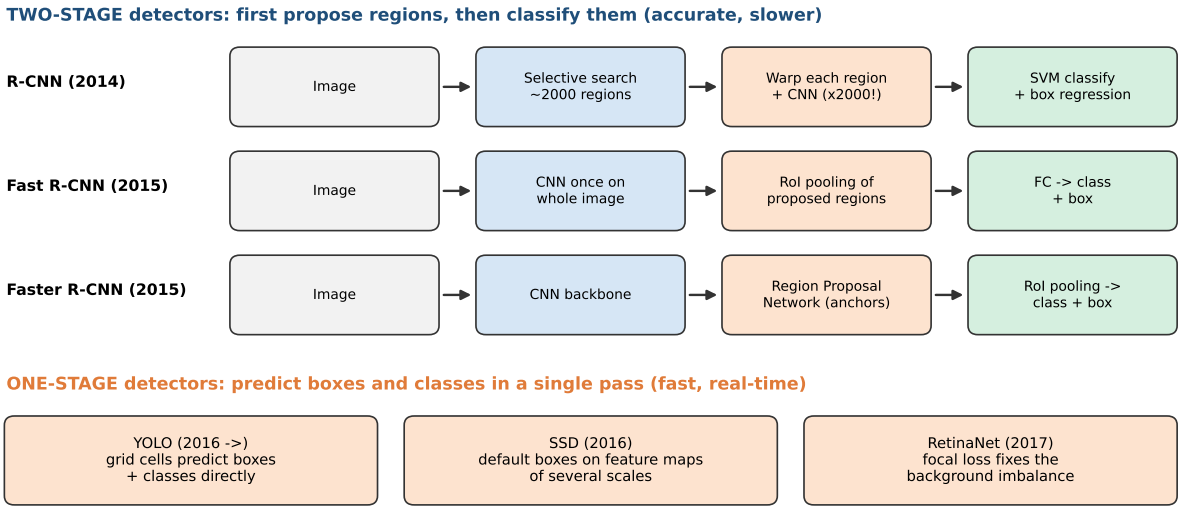


Figure 5.5: Two-stage vs one-stage detectors.

R-CNN (2014) Selective search proposes about 2,000 regions; each is warped and passed through a CNN separately; SVMs classify. Accurate but very slow (thousands of CNN passes per image).

Fast R-CNN (2015) Run the CNN **once** on the whole image, then cut each proposed region out of the feature map with **RoI pooling**. One network predicts class and box. Much faster – but proposals still come from slow selective search.

Faster R-CNN (2015) Replaces selective search with a learned **Region Proposal Network (RPN)** that slides over the feature map and scores anchor boxes. Fully learned end to end; still a strong accuracy baseline.

5.4 One-stage detectors

5.4.1 YOLO – You Only Look Once (2016 onwards)

The idea in plain words

Treat detection as a single regression problem. Divide the image into an $S \times S$ grid; the cell containing an object's centre predicts its box(es), a confidence, and class probabilities – all in **one forward pass**. That makes it real-time.

The original YOLO ran at about 45 frames per second. Later versions added anchors, multi-scale predictions, better backbones and anchor-free heads, and are widely used for real-time detection (CCTV, drones, self-driving, retail).

5.4.2 SSD – Single Shot MultiBox Detector (2016)

One pass like YOLO, but predicts from **feature maps at several scales**: early high-resolution maps catch small objects, deep low-resolution maps catch large ones. Each location has default (anchor) boxes of several shapes.

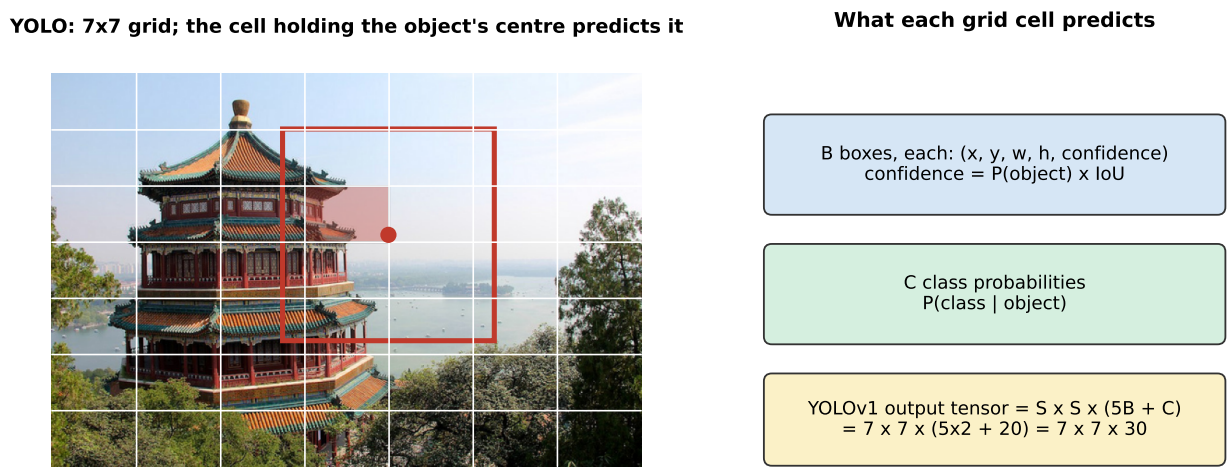


Figure 5.6: YOLOv1's grid idea and its output tensor.

5.4.3 RetinaNet (2017)

One-stage detectors see huge numbers of easy background boxes. **Focal loss** $-(1-p_t)^\gamma \log p_t$ down-weights easy examples so training focuses on hard ones.

	Faster R-CNN	YOLO	SSD
Stages	Two (propose, then classify)	One	One
Speed	Slower	Fastest (real-time)	Fast
Accuracy	High, good on small objects	High in recent versions	Good; weaker on tiny objects
Best for	Accuracy-critical, offline	Video, edge devices	Balanced speed/accuracy

5.5 Image segmentation

5.5.1 FCN – Fully Convolutional Network

Replace the dense layers of a classifier with convolutions and **upsample** back to image size, so the output is a class map for every pixel.

5.5.2 U-Net (2015)

The idea in plain words

A U-shaped encoder–decoder. The **encoder** shrinks the image and learns *what* is there; the **decoder** grows it back to full size to say *where*. **Skip connections** copy fine detail from each encoder level to the matching decoder level, so boundaries stay sharp.

Designed for biomedical images (cells, tumours) and works with relatively few training images. Also the core denoiser inside diffusion models (Chapter 13).

5.5.3 Mask R-CNN (2017)

The idea in plain words

Take Faster R-CNN and add a small third branch that predicts a pixel mask inside each detected box. One model gives class, box and mask for every object (instance segmentation).

It also replaces RoI pooling with **RoIAlign**, which avoids rounding the region coordinates so masks line up precisely with pixels.

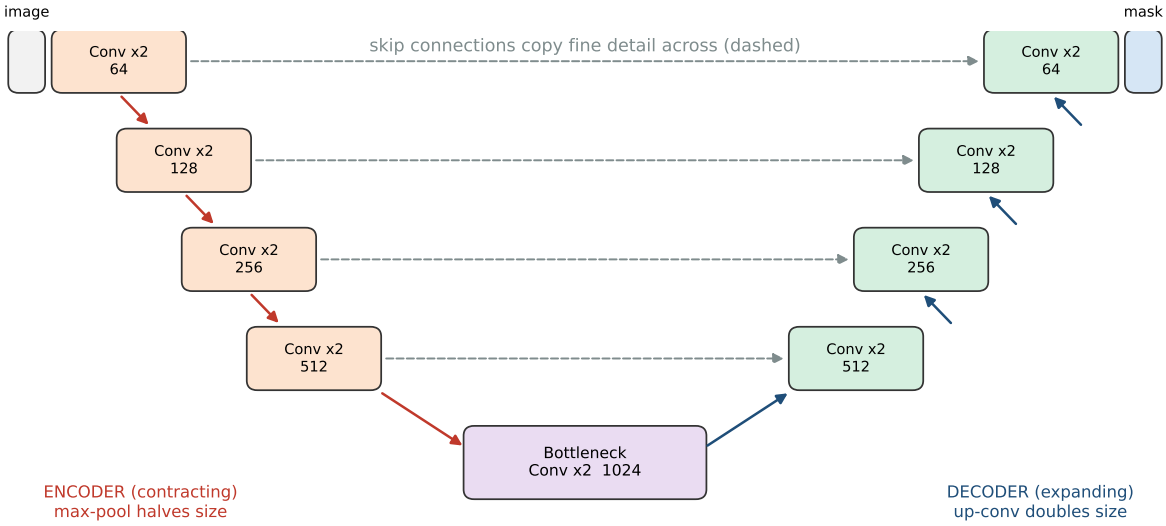


Figure 5.7: U-Net: contracting path, expanding path and skip connections.

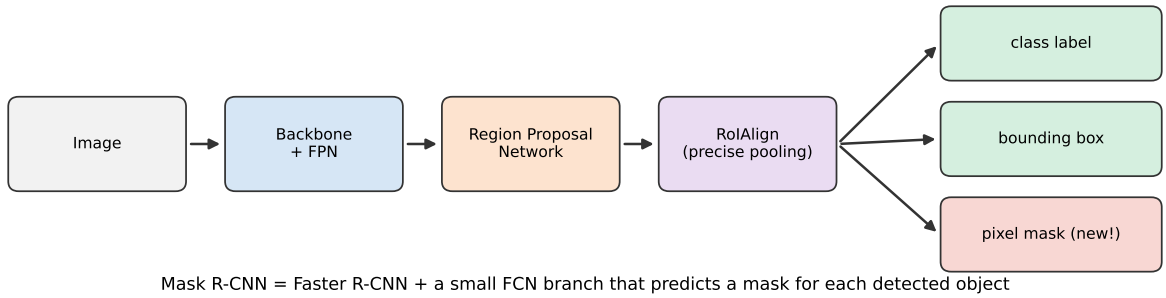


Figure 5.8: Mask R-CNN = Faster R-CNN + mask branch.

5.5.4 Segmentation metrics and losses

Key equations

$$\textbf{IoU (Jaccard)} = \frac{|A \cap B|}{|A \cup B|} \qquad \textbf{Dice} = \frac{2|A \cap B|}{|A| + |B|} \qquad (A = \text{predicted mask}, B = \text{true mask})$$

Dice loss $= 1 -$ Dice is popular when the object covers only a small part of the image (class imbalance).

5.6 CNN visualisation: what is the network looking at?

- **Filter visualisation** – plot first-layer weights; they look like edge and colour detectors.
- **Feature-map (activation) visualisation** – show each layer's outputs for an input image (Figure 3.2).
- **Saliency maps** – gradient of the class score with respect to each pixel: which pixels would change the prediction most.
- **Occlusion sensitivity** – slide a grey patch over the image; where the score drops most is what matters.
- **Grad-CAM** – a coarse heatmap of the regions that drove a class prediction.

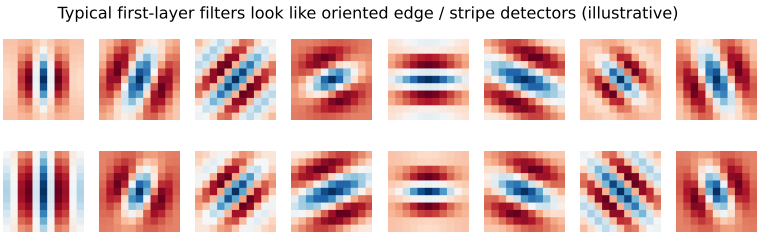


Figure 5.9: First-layer filters typically become oriented edge and stripe detectors (illustrative).

Key equations

$$\textbf{Grad-CAM:} \quad \alpha_k^c = \text{average over pixels of } \frac{\partial y^c}{\partial A^k}, \qquad \text{heatmap} = \text{ReLU}\Big(\textstyle\sum_k \alpha_k^c A^k\Big)$$

$A^k = k$-th feature map of the last conv layer, $y^c =$ score of class c. Feature maps that push the class score up get high weight.

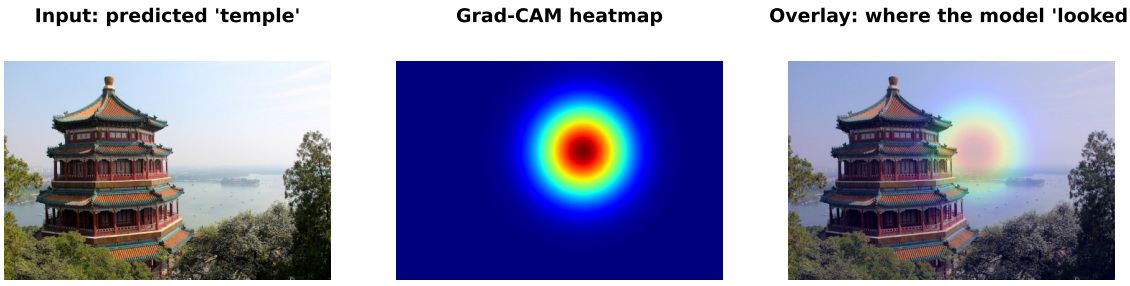


Figure 5.10: Grad-CAM-style heatmap overlay (illustrative). Used to check that a model looks at the right thing – e.g. the lesion, not a ruler or a hospital watermark.

Hands-on in brief

End-to-end computer-vision project. (1) Define the task and metric; (2) collect and label data (classification folders, boxes or masks); (3) split train/val/test; (4) augment; (5) start from a pre-trained backbone (transfer learning); (6) train with early stopping and checkpoints; (7) evaluate – confusion matrix, mAP or IoU/Dice – and inspect Grad-CAM and failure cases; (8) export (SavedModel, ONNX, TFLite) and deploy behind an API; (9) monitor drift and retrain.

CHAPTER 6

Recurrent Neural Networks, LSTM & GRU

6.1 Why sequence models?

Text, speech, stock prices and sensor readings are **sequences**: order matters and the length varies. "Dog bites man" $\neq$ "Man bites dog". A plain ANN or CNN takes a fixed-size input and has no memory of what came before.

6.2 The simple RNN

The idea in plain words

An RNN reads a sequence one step at a time and keeps a **hidden state** h_t – a running summary (memory) of everything seen so far. At each step it combines the new input with the previous memory, **using the same weights at every step**.

Real-life analogy

Reading a novel: you don't restart your understanding at every word; you carry the story so far in your head and update it with each new sentence.

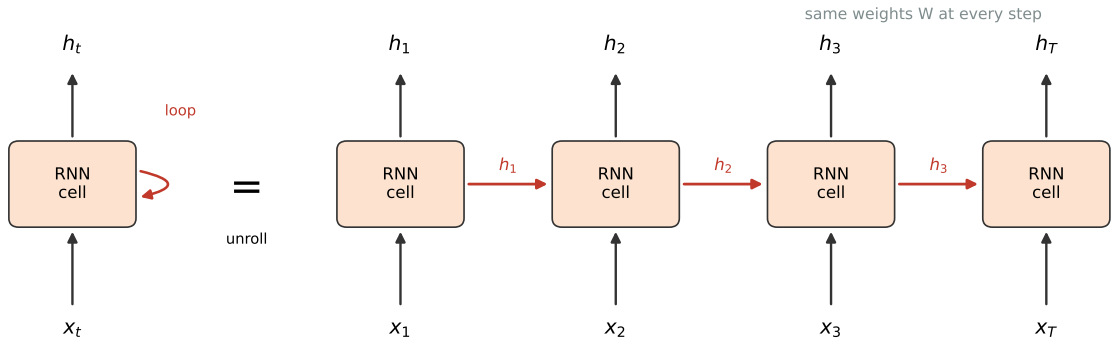


Figure 6.1: An RNN cell with a loop, and the same network "unrolled" through time.

Key equations

$$h_t = \tanh\left(W_{xh}\, x_t + W_{hh}\, h_{t-1} + b_h\right), \qquad \hat{y}_t = W_{hy}\, h_t + b_y$$

x_t = input at time t (e.g. a word vector), h_{t-1} = previous memory, W's are shared across all time steps.

Worked example

Sentiment of "not very good": step 1 reads "not" (memory: something negative may follow), step 2 "very" (intensity), step 3 "good". The final h_3 carries the combined meaning, and a dense layer on h_3 predicts *negative*. A bag-of-words model sees "good" and might say positive.

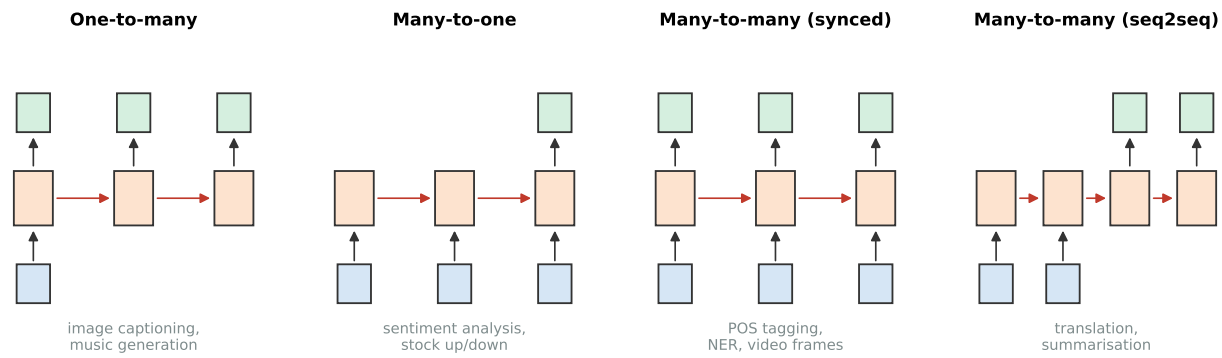


Figure 6.2: Input/output patterns for sequence problems.

6.2.1 Training: backpropagation through time (BPTT)

Unroll the network over all time steps, compute the loss, and backpropagate through the unrolled graph. Because the same weights are reused at each step, their gradients are summed over time. For long sequences, **truncated BPTT** backpropagates only a fixed number of steps.

6.3 The vanishing gradient problem

The idea in plain words

To learn that a word 30 steps ago matters, the error signal has to travel back through 30 steps. At each step it is multiplied by a factor (weights $\times$ tanh slope). If that factor is below 1, the signal shrinks exponentially and **vanishes**; above 1, it **explodes**. A simple RNN therefore has a short memory – roughly the last 5–10 steps in practice.

Key equations

$$\frac{\partial h_t}{\partial h_{t-k}} = \prod_{j=t-k+1}^{t} \frac{\partial h_j}{\partial h_{j-1}} \quad \Longrightarrow \quad \approx (\text{factor})^k$$

e.g. $0.5^{20} \approx 0.000001$ (vanishes); $1.2^{20} \approx 38$ (explodes).

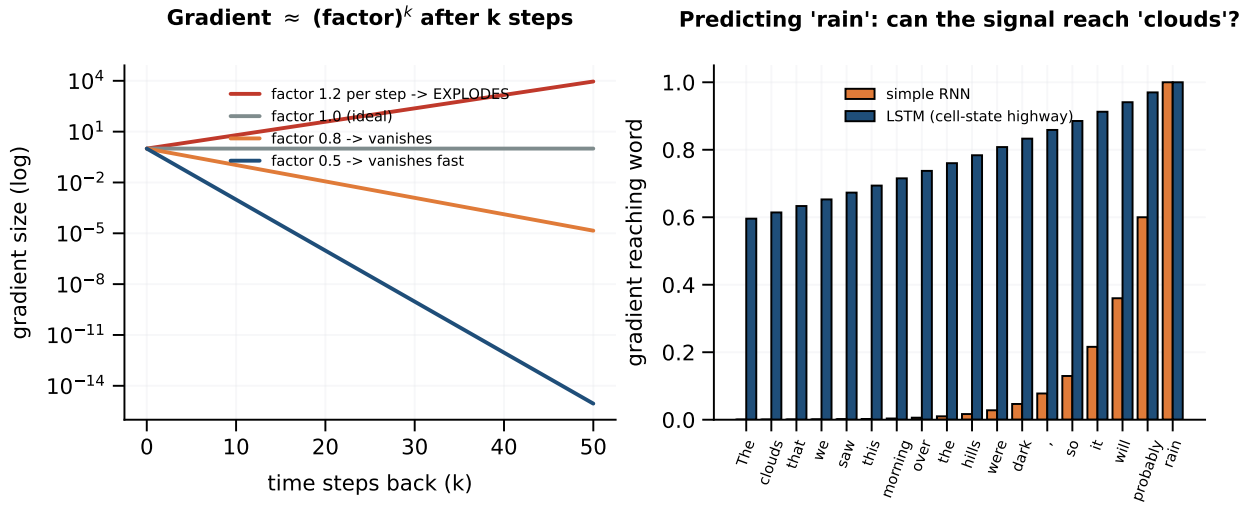


Figure 6.3: Left: repeated multiplication shrinks or blows up the gradient. Right: illustrative – the signal from "clouds" barely reaches "rain" in a simple RNN, but survives in an LSTM.

- **Exploding gradients → gradient clipping**: if the gradient norm exceeds a threshold, scale it down.
- **Vanishing gradients → gated** architectures: LSTM and GRU. (Also ReLU-friendly init, skip connections, and eventually attention.)

6.4 LSTM – Long Short-Term Memory (1997)

The idea in plain words

An LSTM adds a separate **cell state** c_t – a "conveyor belt" that carries information along with only small, controlled changes. Three **gates** (small sigmoid layers outputting 0 to 1) decide what to **forget**, what new information to **write**, and what to **output**. Because the cell state is updated mainly by *addition*, gradients can flow back many steps without vanishing.

Real-life analogy

A notebook during a long meeting: you cross out what is no longer relevant (forget gate), write down important new points (input gate), and when asked, read out only what's relevant now (output gate).

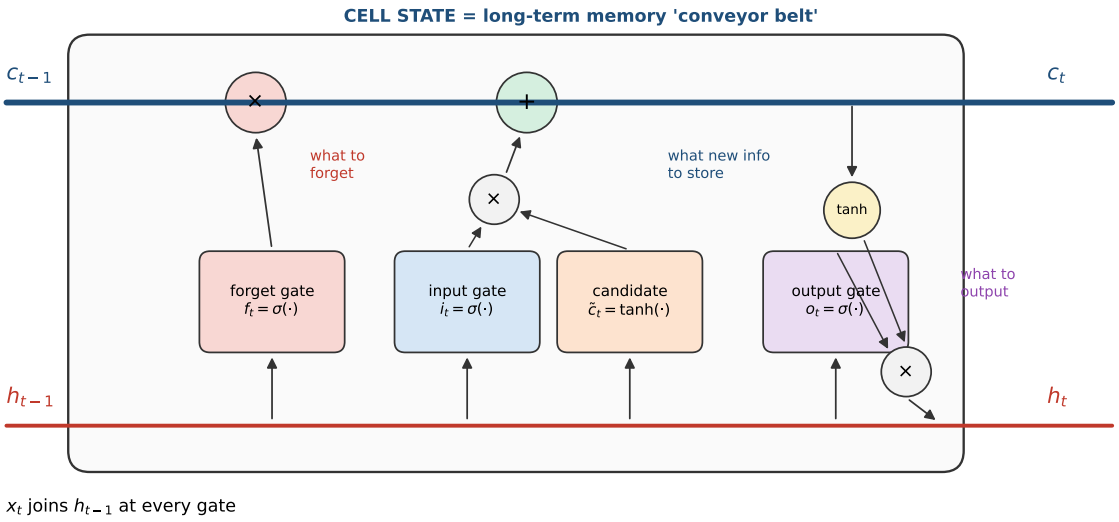


Figure 6.4: Inside an LSTM cell.

Key equations

All gates see $[h_{t-1}, x_t]$ (previous output + current input):

$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$	forget gate: 0 = erase, 1 = keep
$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$	input gate: how much new info to write
$\tilde{c}_t = \tanh(W_c[h_{t-1}, x_t] + b_c)$	candidate new content
$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$	update the long-term memory
$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$	output gate
$h_t = o_t \odot \tanh(c_t)$	new hidden state (short-term output)

Worked example

"*The cats, which were sitting by the window all afternoon, were hungry.*" The LSTM stores "subject is plural" in the cell state when it reads "cats"; the forget gate stays near 1 through the clause, and at "were" the output gate reveals it – so it picks the plural verb.

Parameters: an LSTM layer has 4 sets of weights (3 gates + candidate): $4 \times \left[(n_{\text{input}} + n_{\text{units}}) \times n_{\text{units}} + n_{\text{units}}\right]$. For 10 inputs and 32 units: $4 \times (42 \times 32 + 32) = 5{,}504$.

6.5 GRU – Gated Recurrent Unit (2014)

The idea in plain words

A simplified LSTM: it merges the cell state into the hidden state and uses only **two gates**. The **update gate** decides how much of the old memory to keep vs replace; the **reset gate** decides how much of the past to use when forming the new candidate.

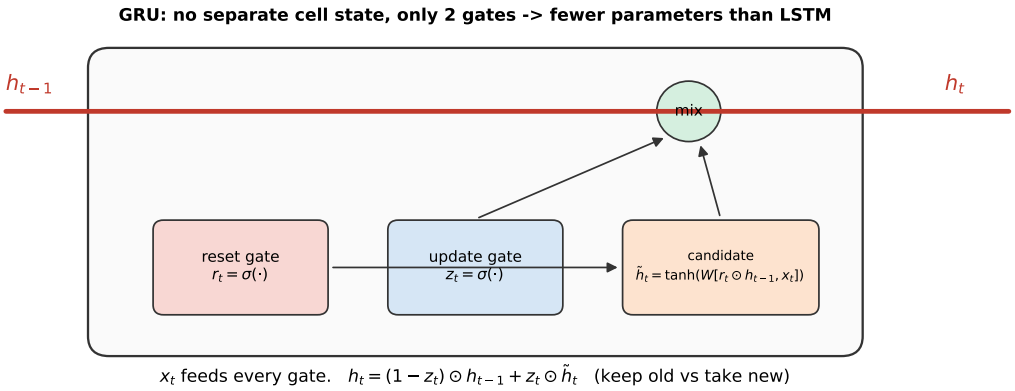


Figure 6.5: GRU cell.

Key equations

$$
\begin{aligned}
z_t &= \sigma(W_z[h_{t-1}, x_t]) && \text{update gate} \\
r_t &= \sigma(W_r[h_{t-1}, x_t]) && \text{reset gate} \\
\tilde{h}_t &= \tanh(W[r_t \odot h_{t-1}, x_t]) && \text{candidate} \\
h_t &= (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t && \text{blend old and new}
\end{aligned}
$$

(Some libraries swap the roles of z_t and $1 - z_t$; the idea is identical.)

6.6 Bidirectional RNNs

The idea in plain words

Run one RNN left-to-right and another right-to-left, then concatenate their states. Each word then knows both its past *and* its future context.

Great for tagging, NER, sentiment and speech recognition when the whole sequence is available. **Not** usable for real-time next-step forecasting or text generation (the future isn't known yet). In Keras: `Bidirectional(LSTM(64))`.

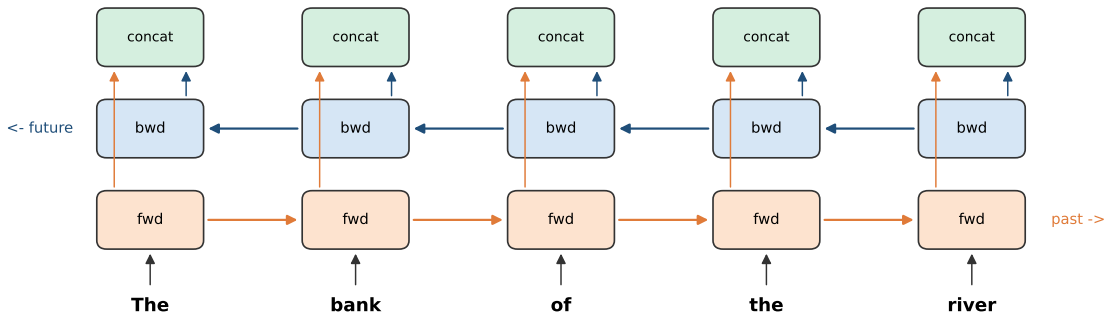


Figure 6.6: "bank" is ambiguous until you read "river" – the backward pass supplies that context.

6.7 Comparison

	Simple RNN	LSTM	GRU
Memory	Hidden state only	Cell state + hidden state	Hidden state only
Gates	None	3 (forget, input, output)	2 (update, reset)
Long dependencies	Poor (vanishing)	Very good	Very good
Parameters	Fewest	Most (4 weight sets)	About 3/4 of LSTM
Speed	Fastest	Slowest	Faster than LSTM
When to use	Very short sequences	Long sequences, default choice	Smaller data, faster training

Where it is used

Language modelling and text generation, sentiment analysis, named-entity recognition, speech recognition, machine translation (before Transformers), time-series forecasting (energy, sales, sensors), anomaly detection in logs, music generation.

Remember

RNNs process tokens **one at a time**, so they can't be parallelised across a sequence and still struggle with very long contexts. These two limits are exactly what the Transformer removed (Chapter 10).

CHAPTER 7

Sequence-to-Sequence Models & Attention

7.1 Encoder–decoder (seq2seq) models

The idea in plain words

Some tasks turn one sequence into another of a *different* length: translation, summarisation, question answering, speech-to-text. A seq2seq model uses two RNNs: an **encoder** that reads the whole input into a **context vector**, and a **decoder** that writes the output one token at a time, starting from that vector.

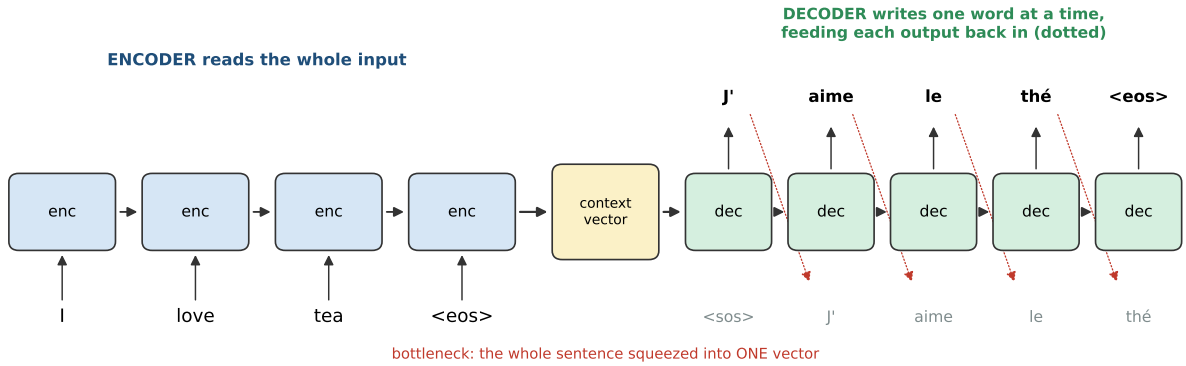


Figure 7.1: Seq2seq translation: "I love tea" → "J'aime le thé".

- The decoder starts with a `<sos>` (start) token and stops when it produces `<eos>`.
- **Teacher forcing** (training): feed the decoder the *true* previous word instead of its own guess – faster, more stable training.
- **Inference**: feed back its own predictions. **Greedy decoding** picks the best word each step; **beam search** keeps the top-k partial sentences and picks the best complete one.

Key equations

The decoder models $P(y_1, \ldots, y_m \mid x) = \prod_t P(y_t \mid y_{<t}, c)$ $\quad$ (c = context vector).

Watch out

The bottleneck problem: the entire input – however long – must be squeezed into one fixed-size vector. Quality drops sharply for long sentences. Attention was invented to fix this.

7.2 The attention mechanism

The idea in plain words

Instead of relying on one summary vector, let the decoder **look back at all the encoder states** at every step and focus on the ones most relevant to the word it is about to write. When translating "chat", it looks mostly at "cat".

Real-life analogy

A human translator doesn't memorise a whole paragraph and then write; they glance back at the relevant part of the source for each phrase.

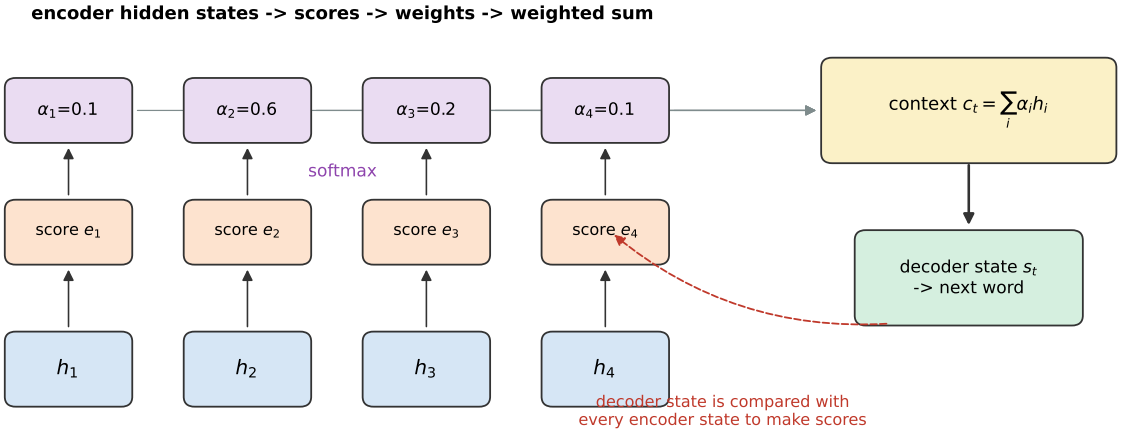


Figure 7.2: Attention at one decoder step: score every encoder state, turn scores into weights with softmax, take the weighted sum.

Key equations

At decoder step t, with encoder states $h_1, \ldots, h_n$ and decoder state s_t:

1. **Scores:** $e_{t,i} = \text{score}(s_t, h_i)$ (how relevant is input i?)
2. **Weights:** $\alpha_{t,i} = \dfrac{\exp(e_{t,i})}{\sum_j \exp(e_{t,j})}$ (softmax: positive, sum to 1)
3. **Context:** $c_t = \sum_i \alpha_{t,i}\, h_i$ (weighted average of encoder states)

Bahdanau (additive, 2014): score $= v^\top \tanh(W_1 s_{t-1} + W_2 h_i)$ **Luong (multiplicative, 2015):** score $= s_t^\top W h_i$ or simply $s_t^\top h_i$ (dot product)

Benefits: handles long inputs, learns alignment automatically, and is **interpretable** (you can plot the weights). Attention is the core idea behind the Transformer, which drops the RNN altogether.

Hands-on in brief

Time-series forecasting with LSTM.

1. Scale the series (e.g. MinMaxScaler fitted on training data only).
2. Build **sliding windows**: input = last n values (e.g. 30), target = next value (or next k values).
3. Reshape to 3-D `(samples, timesteps, features)` – e.g. (970, 30, 1).
4. Model: `LSTM(64)` → `Dense(1)`; loss MSE, optimiser Adam, early stopping.
5. Split **by time** (never shuffle across the split); evaluate with MAE/RMSE; invert the scaling before reporting.
6. Compare against a naive baseline ("tomorrow = today"). If the LSTM can't beat it, it isn't learning anything useful.

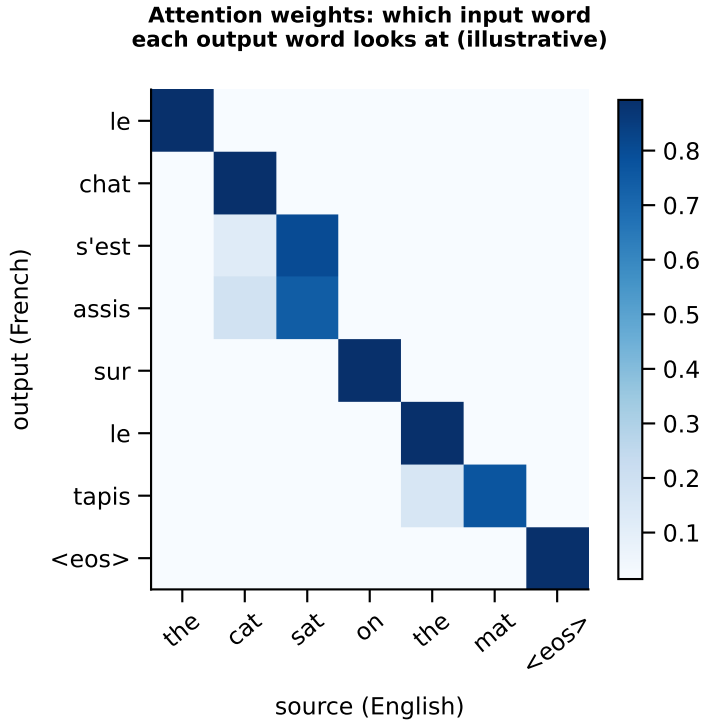


Figure 7.3: Attention weights form a soft alignment between output and input words (illustrative). Word-order differences such as "s'est assis" show up as off-diagonal weights.

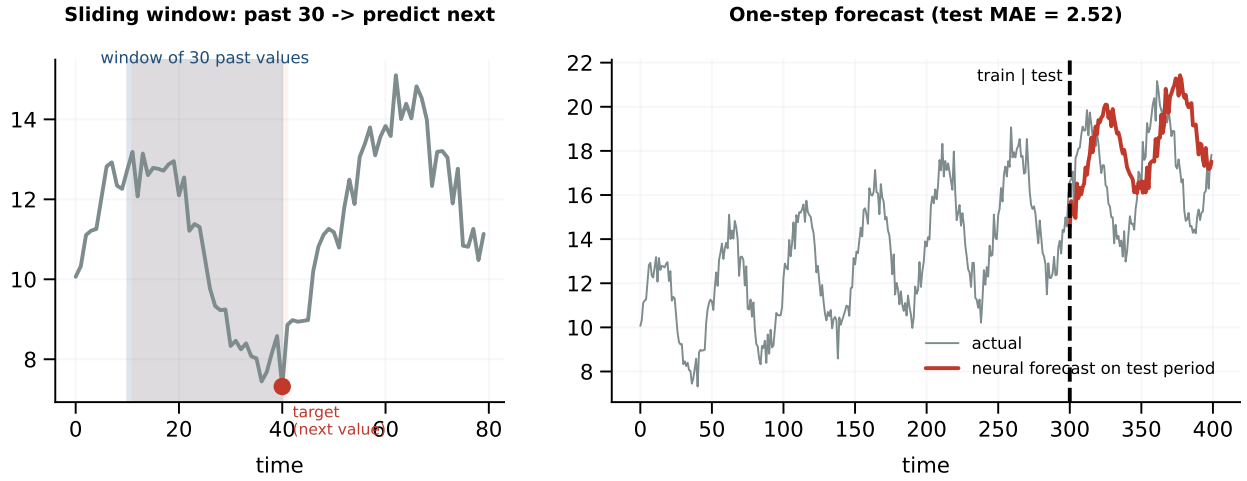


Figure 7.4: The windowing idea and a one-step-ahead forecast on a trend + seasonal series. The forecast here comes from a small neural network trained on the same windows an LSTM would use.

CHAPTER 8

Generative Models: Autoencoders, GANs & VAEs

8.1 Discriminative vs generative

- **Discriminative** models learn boundaries: *is this a cat or a dog?* – $P(y \mid x)$.
- **Generative** models learn what the data itself looks like, so they can **create new samples**: *draw a new cat* – $P(x)$.

Main families: autoencoders/VAEs, GANs, autoregressive models (GPT-style, one token at a time), flow models, and diffusion models (Chapter 13).

8.2 Autoencoders

The idea in plain words

Train a network to **copy its input to its output through a narrow bottleneck**. To succeed it must learn a compact code that keeps only the important information.

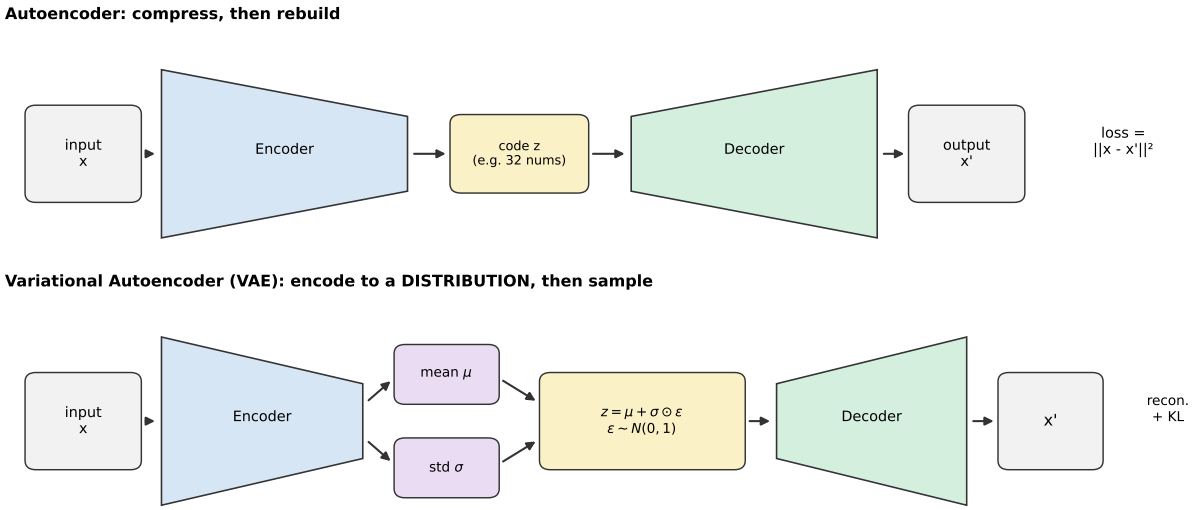


Figure 8.1: Top: an autoencoder. Bottom: a variational autoencoder encodes to a distribution.

Uses: dimensionality reduction, denoising (train on noisy inputs, clean targets), anomaly detection (high reconstruction error = unusual). A plain autoencoder is a poor generator: random points in its code space often decode to garbage.

8.3 GANs – Generative Adversarial Networks (2014)

The idea in plain words

Two networks play a game. The **generator** turns random noise into fake samples. The **discriminator** tries to tell real samples from fakes. The generator improves by learning to fool the discriminator; the discriminator improves by catching it. In the ideal end state the fakes are indistinguishable from real data.

Real-life analogy

A counterfeiter (generator) and a police detective (discriminator). Each gets better because of the other, until the fake notes look exactly like real ones.

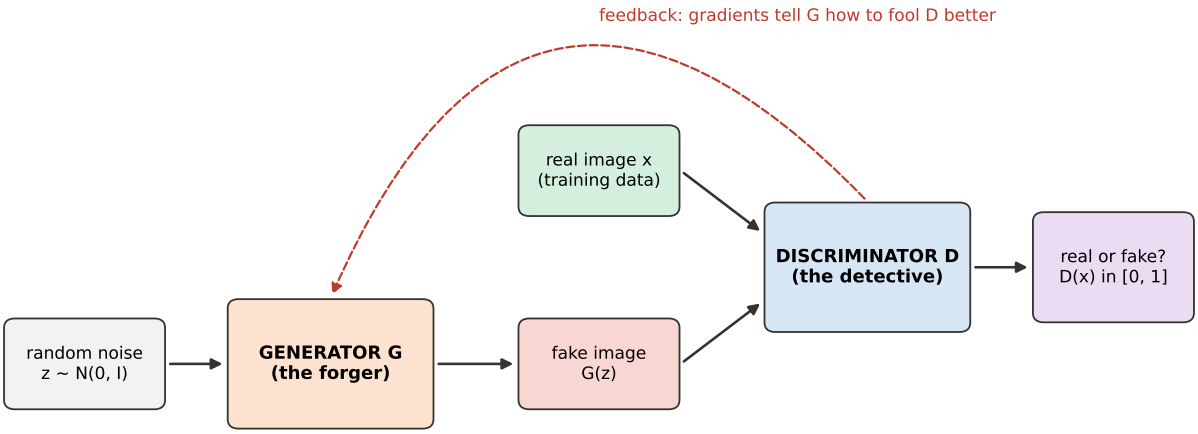


Figure 8.2: The GAN set-up.

Key equations

Minimax game: $\min_G \max_D \; \mathbb{E}_{x\sim\text{data}}[\log D(x)] + \mathbb{E}_{z\sim p(z)}[\log(1 - D(G(z)))]$

- D wants $D(x) \to 1$ for real and $D(G(z)) \to 0$ for fake.
- G wants $D(G(z)) \to 1$. In practice G maximises $\log D(G(z))$ ("non-saturating" loss) because it gives stronger gradients early on.
- At the ideal equilibrium the generator matches the data and $D(x) = \frac{1}{2}$ everywhere.

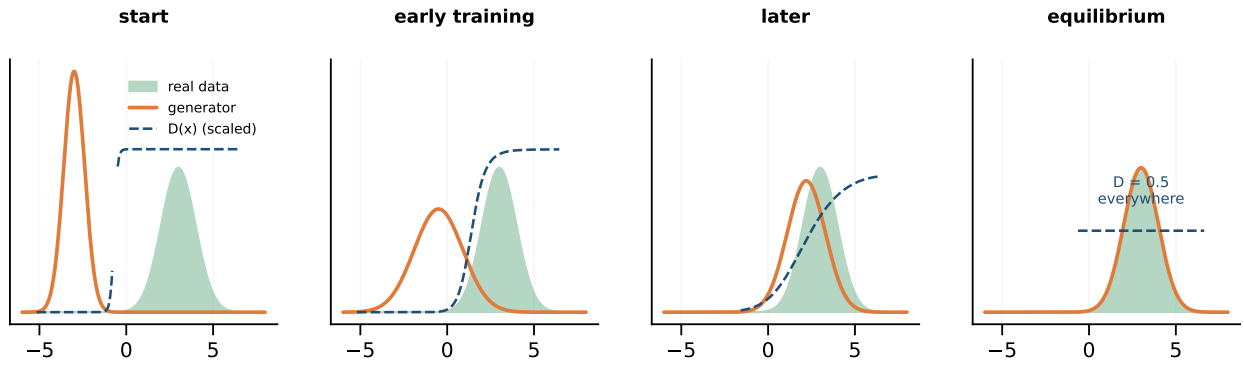


Figure 8.3: 1-D illustration: the generator's distribution (orange) moves towards the real data (green); the best discriminator (dashed) flattens to 0.5 when they match.

Training loop: (1) update D on a batch of real images (label 1) and fakes (label 0); (2) update G through D, with D frozen, so the fakes get classified as real; repeat.

8.3.1 Problems with GANs

- **Mode collapse** – the generator finds a few outputs that fool D and produces only those (low diversity).
- **Instability / non-convergence** – the two players oscillate instead of settling.
- **Vanishing gradients** – if D becomes too good, G gets almost no signal.

- **Hard to evaluate** – no single loss tells you quality. FID (Fréchet Inception Distance, lower is better) and Inception Score are common.

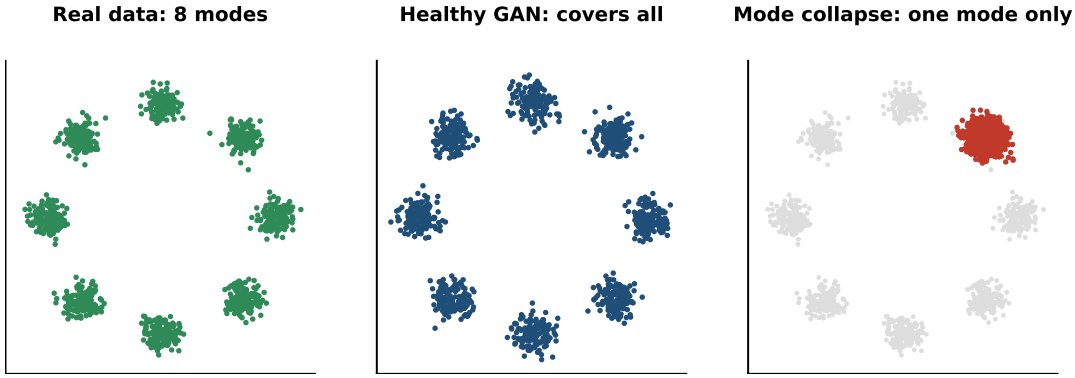


Figure 8.4: Mode collapse: the generator covers one of eight modes.

8.4 Advanced GAN variants

Variant	Year	Key idea / use
DCGAN	2015	Convolutional G and D, batch norm, no pooling – the standard image-GAN recipe
Conditional GAN	2014	Feed a label to G and D: "generate a digit 7"
Pix2Pix	2017	Paired image-to-image translation (sketch → photo, map → satellite)
CycleGAN	2017	*Unpaired* translation (horse ↔ zebra) using cycle-consistency: $F(G(x)) \approx x$
WGAN / WGAN-GP	2017	Wasserstein distance with a "critic"; much more stable training, meaningful loss
ProGAN	2017	Grow resolution progressively during training
StyleGAN (1–3)	2018–21	Style-based generator; photorealistic faces, controllable style per layer

8.5 VAEs – Variational Autoencoders (2013)

The idea in plain words

A VAE's encoder outputs a **distribution** (a mean μ and spread σ) instead of a single point. We sample a code from it and decode. A second loss term pulls all these distributions towards a standard normal, so the code space is **smooth and continuous** – any random point decodes to something sensible, and you can interpolate between samples.

Key equations

Reparameterisation trick: $z = \mu + \sigma \odot \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, I)$ (randomness moved outside so gradients can flow)

Loss = reconstruction error + KL divergence: $\mathcal{L} = \|x - \hat{x}\|^2 + D_{KL}(\mathcal{N}(\mu, \sigma^2) \,\|\, \mathcal{N}(0,1))$
with $D_{KL} = -\frac{1}{2}\sum(1 + \log\sigma^2 - \mu^2 - \sigma^2)$.

VAEs train stably and give a meaningful latent space but produce **blurrier** images than GANs. Their encoder–decoder idea lives on inside Stable Diffusion, which uses a VAE to compress images into a small latent space.

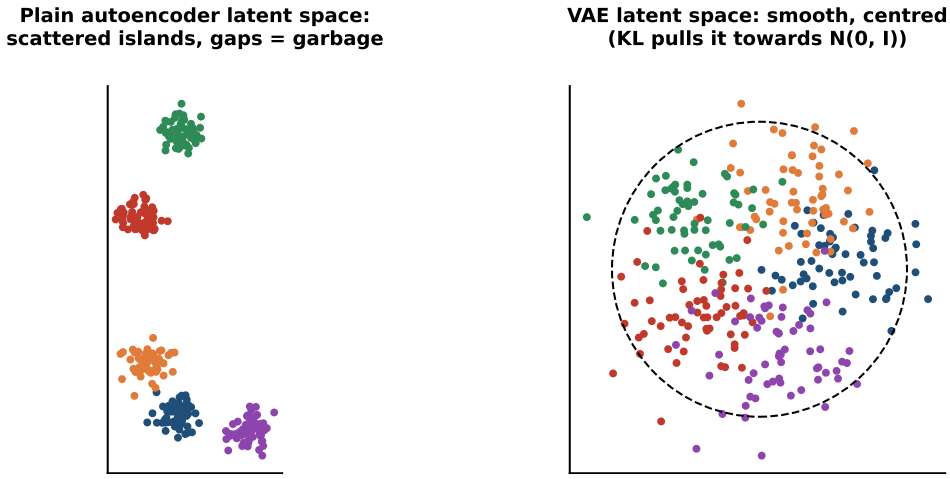


Figure 8.5: Illustrative latent spaces: a VAE's KL term makes the space compact and continuous.

8.6 GANs, VAEs and modern GenAI

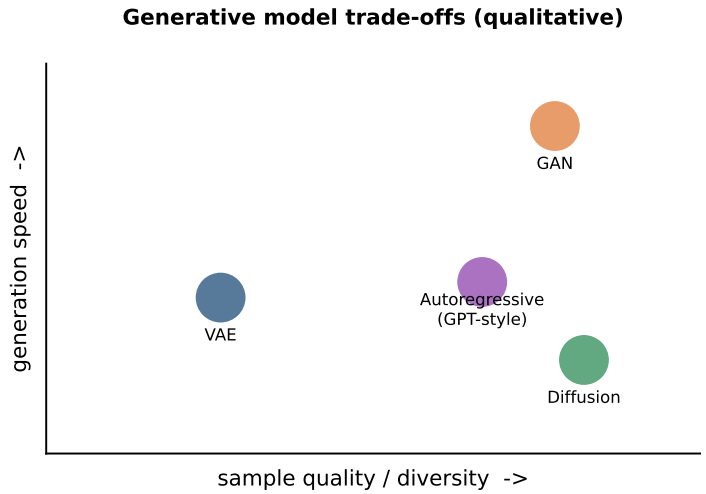


Figure 8.6: Qualitative trade-offs between generative families.

	GAN	VAE	Diffusion (modern GenAI)
Training	Adversarial, unstable	Stable (one loss)	Stable (predict the noise)
Image quality	Sharp	Blurry	Sharp and diverse
Diversity	Risk of mode collapse	Good	Very good
Speed to generate	One pass – fast	One pass – fast	Many denoising steps – slower
Text control	Limited	Limited	Excellent (text prompts)
Examples	StyleGAN, CycleGAN	VAE, VQ-VAE	Stable Diffusion, DALL-E 2/3, Imagen

Today's GenAI combines these ideas: a **VAE** compresses images, a **diffusion** model generates in that compressed space, and a **Transformer** text encoder steers it. For text, **autoregressive Transformers** (LLMs) dominate. GANs remain useful where single-pass speed matters (super-resolution, real-time filters).

CHAPTER 9

NLP: Classical Text Processing & Word Embeddings

9.1 What is NLP?

Natural Language Processing teaches computers to understand and generate human language. Common tasks: text classification (spam, sentiment), named-entity recognition (people, places, dates), part-of-speech tagging, machine translation, question answering, summarisation, chatbots. Language is hard because of ambiguity ("bank"), context, sarcasm, spelling variety, and many



Figure 9.1: How NLP evolved.

languages and scripts.

9.2 Classical text processing

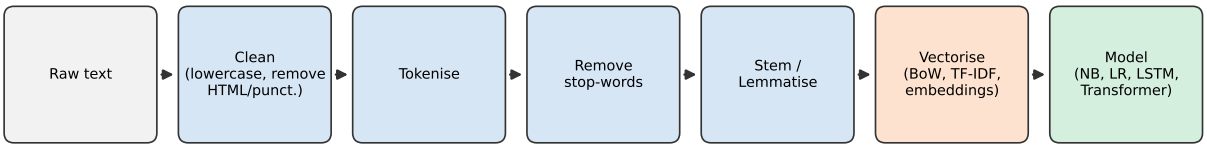


Figure 9.2: A classical NLP pipeline.

Step	What it does	Example
Cleaning	Lower-case, strip HTML, URLs, punctuation, extra spaces	"Great!!! " → "great"
Tokenisation	Split text into tokens (words, sub-words)	"I'm happy" → [I, 'm, happy]
Stop-word removal	Drop very common words with little meaning	remove "the", "is", "a"
Stemming	Chop word endings by rules (fast, crude)	studies, studying → "studi"
Lemmatisation	Reduce to dictionary form using grammar	studies, studying → "study"; better → "good"
POS tagging	Label grammatical role	"run" = verb or noun
NER	Find named entities	"Rahul joined LTIMindtree in Bengaluru"
N-grams	Groups of n consecutive words	"not good" as one feature (bigram)

Watch out

Don't blindly remove stop-words for sentiment: removing "not" turns "not good" into "good". Transformers usually need *no* stemming or stop-word removal – they use their own sub-word tokenisers.

9.3 Turning text into numbers

9.3.1 One-hot and Bag-of-Words (BoW)

One-hot: each word is a vector with a single 1 (length = vocabulary size). Bag-of-Words: count how many times each vocabulary word appears in a document. Simple and effective for classification, but it **ignores word order** and treats "good" and "great" as unrelated.

9.3.2 TF-IDF

The idea in plain words

Weight a word higher if it appears often *in this document* but rarely *across all documents*. Common words like "the" get weight near zero; distinctive words stand out.

Key equations

$$\text{TF-IDF}(t, d) = \text{TF}(t, d) \times \log \frac{N}{\text{DF}(t)}$$

TF = count of term t in document d (often divided by document length), N = number of documents, DF = documents containing t. (Libraries add small smoothing terms.)

Worked example

Three documents: D1 "the cat sat", D2 "the dog sat", D3 "the cat ran".
"the" is in all 3: IDF $= \ln(3/3) = 0 \Rightarrow$ weight 0 everywhere.
"cat" is in 2: IDF $= \ln(3/2) = 0.405$; in D1, TF $= 1/3 \Rightarrow$ TF-IDF $= 0.135$.
"dog" is in 1: IDF $= \ln 3 = 1.099$; in D2, TF-IDF $= 0.366$ – the most distinctive word.

9.4 Word embeddings

The idea in plain words

Represent each word as a short, dense vector (e.g. 100–300 numbers) learned from text, so that words used in similar contexts get similar vectors. "You shall know a word by the company it keeps."

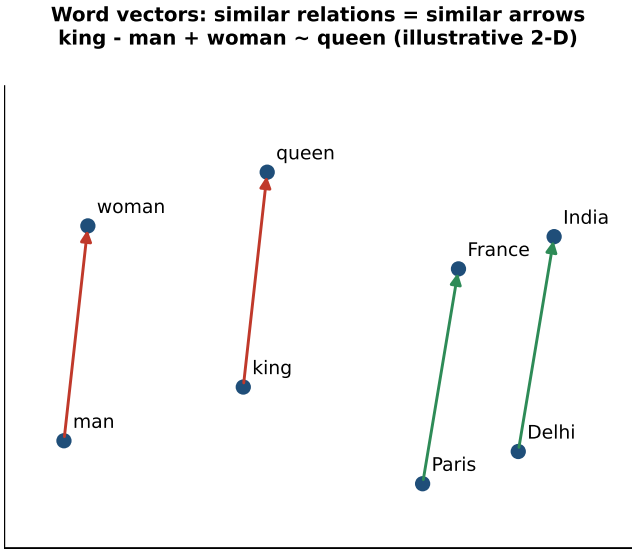


Figure 9.3: Embeddings capture relationships as directions (illustrative 2-D projection).

Key equations

Cosine similarity: $\cos(\mathbf{a}, \mathbf{b}) = \dfrac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \, \|\mathbf{b}\|}$ (1 = same direction, 0 = unrelated)

Analogy: $\vec{\text{king}} - \vec{\text{man}} + \vec{\text{woman}} \approx \vec{\text{queen}}$

9.4.1 Word2Vec (2013)

A shallow network trained on a fake task; the learned weights become the embeddings.

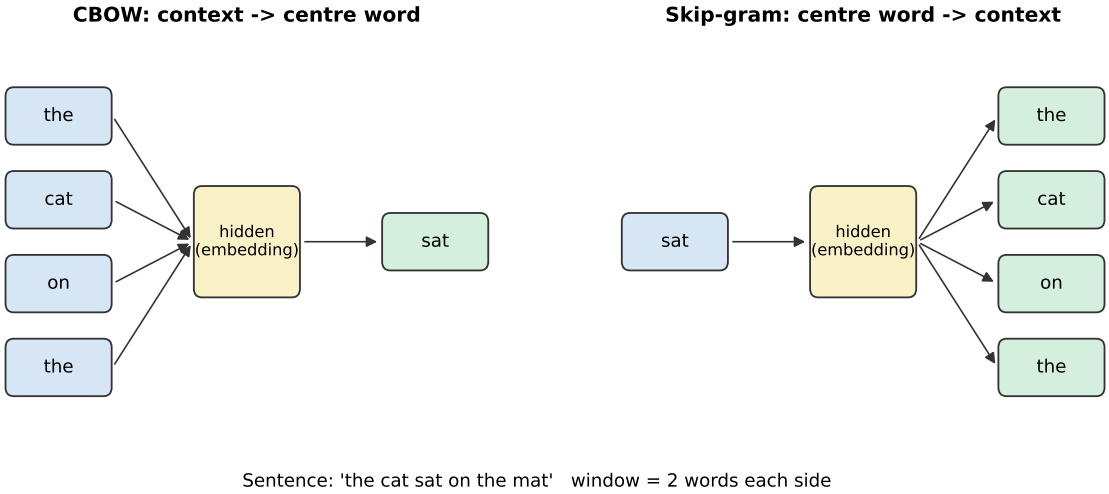


Figure 9.4: CBOW predicts the centre word from its context; skip-gram predicts the context from the centre word.

- **CBOW** – faster, good for frequent words. **Skip-gram** – better for rare words and small data.
- **Negative sampling** – instead of a softmax over the whole vocabulary, train the model to tell real (word, context) pairs from a few random "negative" pairs. Much faster.

9.4.2 GloVe (2014) and FastText (2016)

- **GloVe** – learns vectors from global word co-occurrence counts across the whole corpus.
- **FastText** – builds a word from character n-grams ("play" + "ing"), so it can create vectors for unseen or misspelled words; good for morphologically rich languages.

Watch out

These are **static** embeddings: "bank" gets one vector whether it means a river bank or a money bank. Contextual embeddings from ELMo, BERT and GPT fix this by producing a different vector for each use.

9.5 Classical NLP pipelines

Pipeline	Good for
TF-IDF + Logistic Regression / Linear SVM	Strong, fast baseline for classification (spam, topic, sentiment)
BoW / TF-IDF + Multinomial Naive Bayes	Very fast, small data
Average word vectors + classifier	Some semantic similarity, still order-blind
Embedding layer + LSTM/GRU	Order-aware; sentiment, NER, sequence tagging
Pre-trained Transformer (BERT) + fine-tuning	Today's default for best accuracy

9.6 From sequence models to Transformers

RNN/LSTM encoders read text word by word, so training can't be parallelised across positions, long-range links are still weak, and a seq2seq bottleneck limits quality. Attention (Chapter 7) fixed the bottleneck. The 2017 paper *Attention Is All You Need* asked: **what if we keep the attention and remove the RNN entirely?** That is the Transformer.

CHAPTER 10

Transformers

10.1 Attention before Transformers – a quick recap

In RNN seq2seq models, attention was an *add-on*: the decoder RNN attended over encoder RNN states (Bahdanau additive or Luong dot-product scores). It solved the bottleneck, but the model still processed words one by one. The Transformer (Vaswani et al., 2017) uses attention for everything: **every word attends to every other word in the same sentence – self-attention – in parallel.**

10.2 Self-attention

The idea in plain words

For each word, ask: *which other words in this sentence help me understand myself?* In "The animal didn't cross the street because it was tired", "it" should pay attention to "animal". Self-attention computes such a relevance score between every pair of words and builds a new, context-aware vector for each word as a weighted mix of the others.

Real-life analogy

A library search: your **query** is what you're looking for; each book has a **key** (its title/label) and a **value** (its content). You compare your query with all keys, then read a blend of the contents, weighted by how well each key matched.

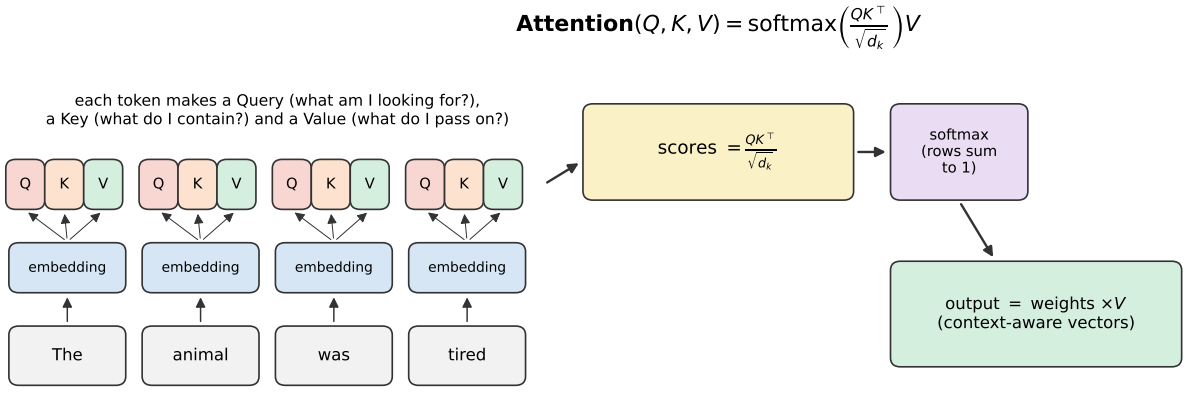


Figure 10.1: Each token produces a Query, Key and Value by multiplying its embedding with three learned matrices.

Key equations

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V, \qquad \text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

- $QK^\top$ – similarity between every query and every key (an $n \times n$ table for n tokens).
- $\div\sqrt{d_k}$ – scaling, so large dot products don't push the softmax into regions with tiny gradients.
- softmax – each row becomes weights that sum to 1.
- $\times V$ – each output is a weighted average of the value vectors.

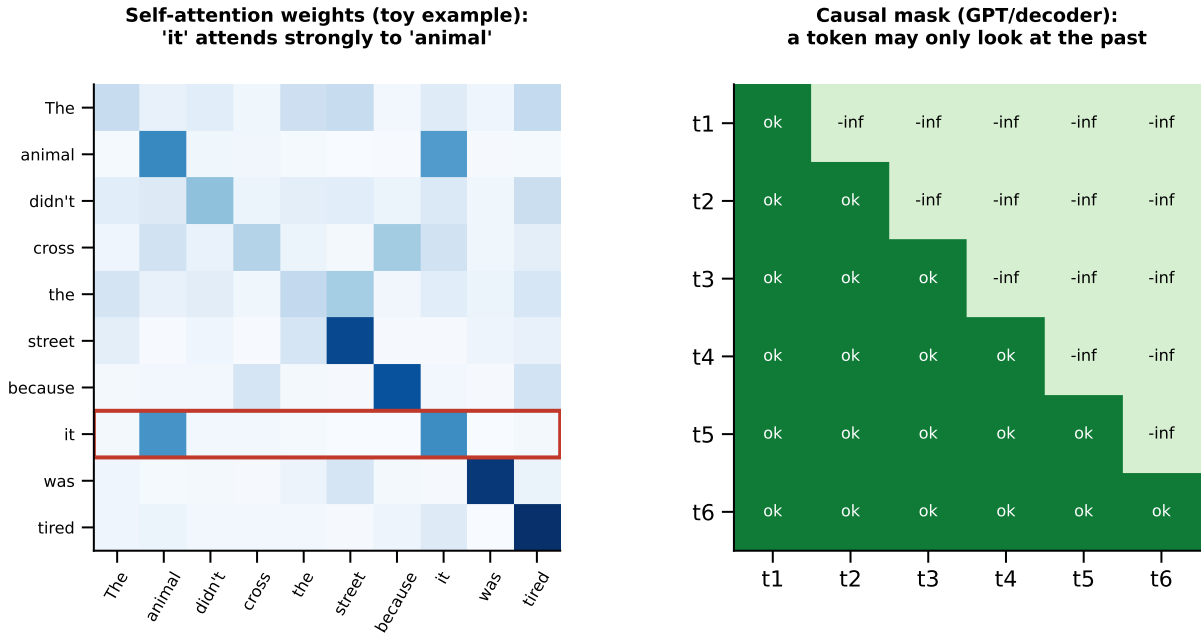


Figure 10.2: Left: a toy self-attention matrix computed with the formula above. Right: the causal mask used in decoders – future positions get $-\infty$ before the softmax, so their weight becomes 0.

Worked example

Two tokens, $d_k = 4$. Query of "it" $q = (1, 0, 1, 0)$; keys: "animal" $(1, 0, 1, 0)$, "street" $(0, 1, 0, 0)$. Scores: $q \cdot k = 2$ and 0; scaled by $\sqrt{4} = 2$: 1 and 0. Softmax: $e^1/(e^1 + e^0) = 0.73$ and 0.27. So "it" takes 73% of its new meaning from "animal".

10.3 Multi-head attention

The idea in plain words

One attention pattern can't capture everything. Run several attention "heads" in parallel, each with its own W_Q, W_K, W_V; one may track grammar, another which noun a pronoun refers to, another nearby words. Concatenate their outputs and mix them with one more matrix.

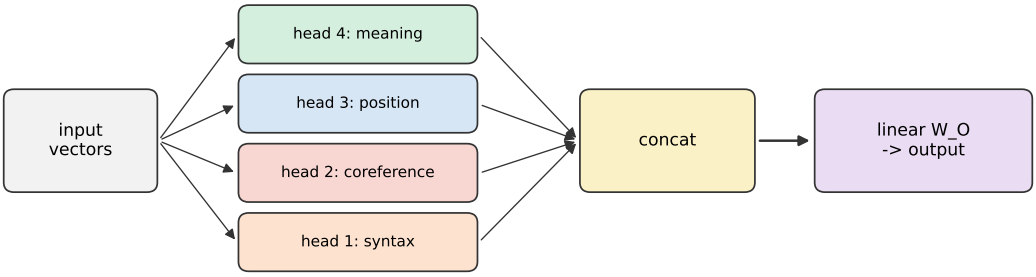


Figure 10.3: Multi-head attention (head roles are illustrative – heads learn their own specialities).

Key equations

$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)\, W_O$. Original Transformer: $d_{\text{model}} = 512$, $h = 8$ heads of size 64.

10.4 Positional encoding

Self-attention treats the input as a *set* – shuffling the words gives the same result. To give the model word order, add a **position vector** to each token embedding.

Key equations

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d}}\right), \qquad PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d}}\right)$$

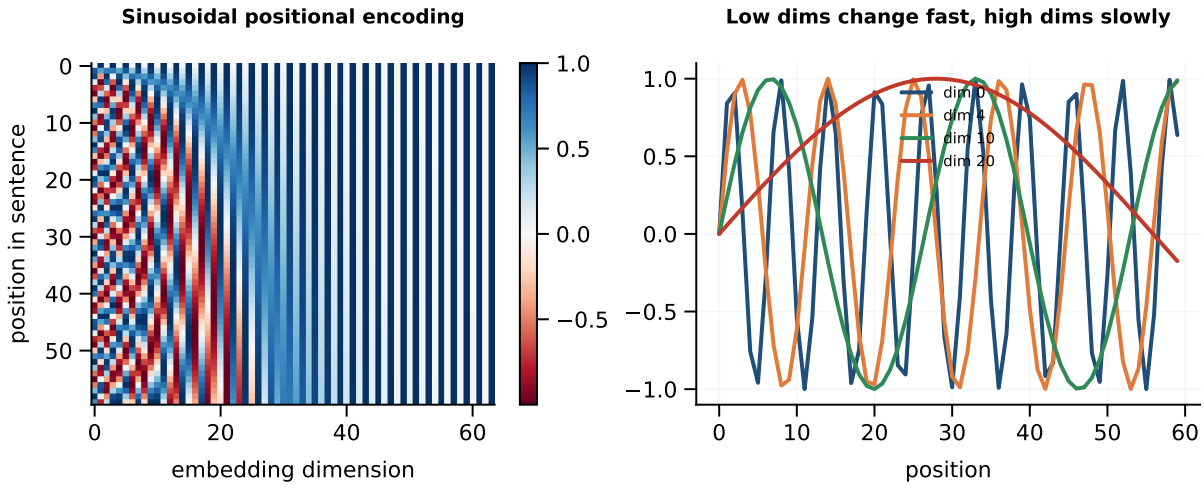


Figure 10.4: Real sinusoidal encodings: each position gets a unique pattern of waves of different frequencies.

Many modern models use *learned* position embeddings (BERT, GPT-2) or rotary embeddings (RoPE, used in Llama-style models).

10.5 The full Transformer architecture

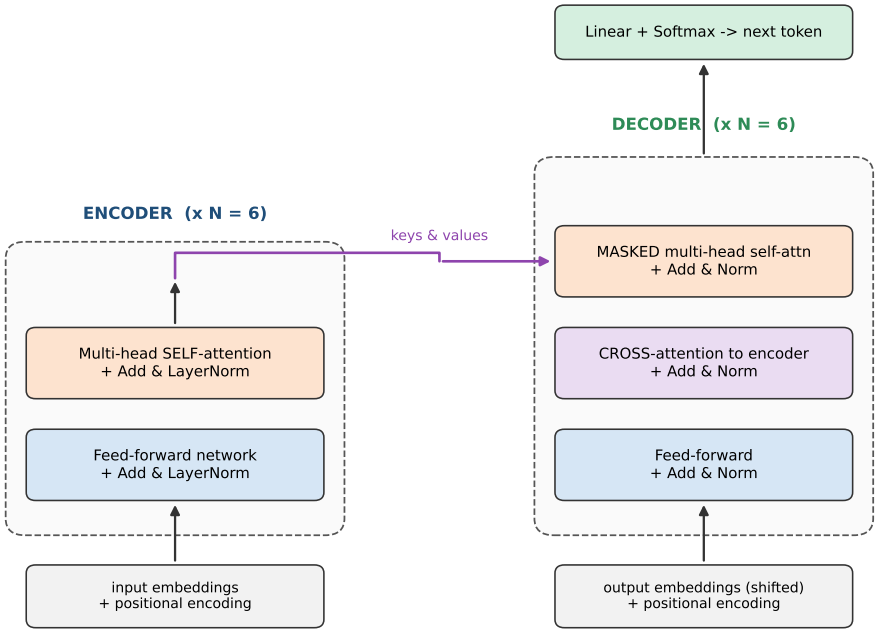


Figure 10.5: Encoder–decoder Transformer.

- **Encoder block** (repeated N times; 6 in the original): multi-head self-attention → add & layer-norm → position-wise feed-forward network → add & layer-norm.
- **Decoder block**: *masked* self-attention (can't see future words) → **cross-attention** (queries from the decoder, keys and values from the encoder output) → feed-forward. Each sub-layer has add & norm.
- **Residual connections** ("add") and **layer normalisation** keep deep stacks trainable – the same idea as ResNet.
- **Feed-forward network**: $\mathrm{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$, applied to each position separately (inner size 2048 in the original).
- **Output**: linear layer + softmax over the vocabulary → probability of the next token.

10.6 Why Transformers work so well

	RNN / LSTM	Transformer
Processing	One token after another	All tokens in parallel (fast on GPUs)
Path between distant words	$O(n)$ steps – signal fades	1 step – direct attention
Long-range context	Weak	Strong
Scaling with data/compute	Limited	Keeps improving with size (scaling laws)
Cost	Linear in length	Attention is quadratic in length (n^2)

Parallel training made it practical to train on internet-scale text, which, combined with transfer learning (pre-train once, fine-tune many times), produced BERT, GPT and today's LLMs.

10.7 Three families of Transformers

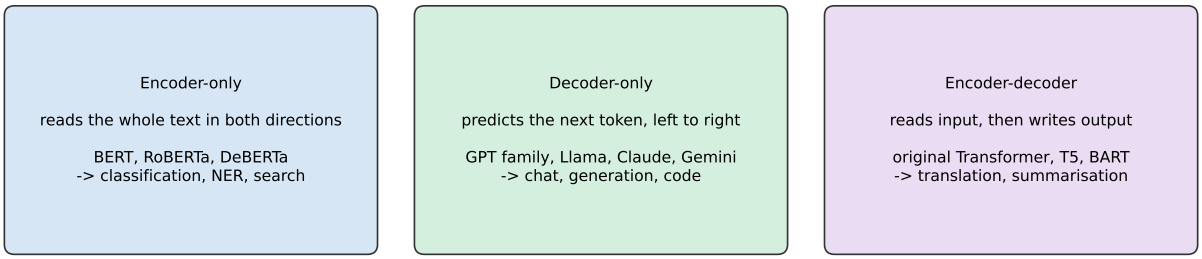


Figure 10.6: Encoder-only, decoder-only and encoder–decoder models.

CHAPTER 11

BERT: Architecture, Pre-training & Variants

11.1 What is BERT?

The idea in plain words

BERT (**B**idirectional **E**ncoder **R**epresentations from **T**ransformers, Google, 2018) is a stack of Transformer **encoders** pre-trained on huge amounts of text to understand language. Unlike left-to-right models, every word sees the words on **both sides** at once. You then **fine-tune** it with a small labelled dataset for your task.

Real-life analogy

BERT is like a graduate who has read a whole library (pre-training). Teaching them a specific job – sorting complaints, finding names in contracts – needs only a short training course (fine-tuning).

Model	Layers	Hidden size	Attention heads	Parameters
BERT-Base	12	768	12	≈110M
BERT-Large	24	1024	16	≈340M

Pre-trained on BooksCorpus and English Wikipedia; WordPiece tokeniser with about 30,000 sub-word tokens ("playing" → "play", "##ing").

11.2 Input representation

- **[CLS]** at the start – its final vector summarises the whole input and is used for classification.
- **[SEP]** separates sentence A from sentence B and ends the input.
- Each token's input = **token embedding** + **segment embedding** (sentence A or B) + **position embedding** (learned).

11.3 Pre-training tasks

Key equations

1. Masked Language Model (MLM): pick 15% of tokens; of these, 80% become [MASK], 10% a random token, 10% stay unchanged. Predict the originals: $\mathcal{L}_{\text{MLM}} = -\sum_{i \in \text{masked}} \log P(x_i \mid \text{context})$

2. Next Sentence Prediction (NSP): is sentence B the real next sentence after A (50%) or a random one (50%)? Uses the [CLS] vector.

The 80/10/10 trick stops the model from relying on seeing [MASK], which never appears during fine-tuning.

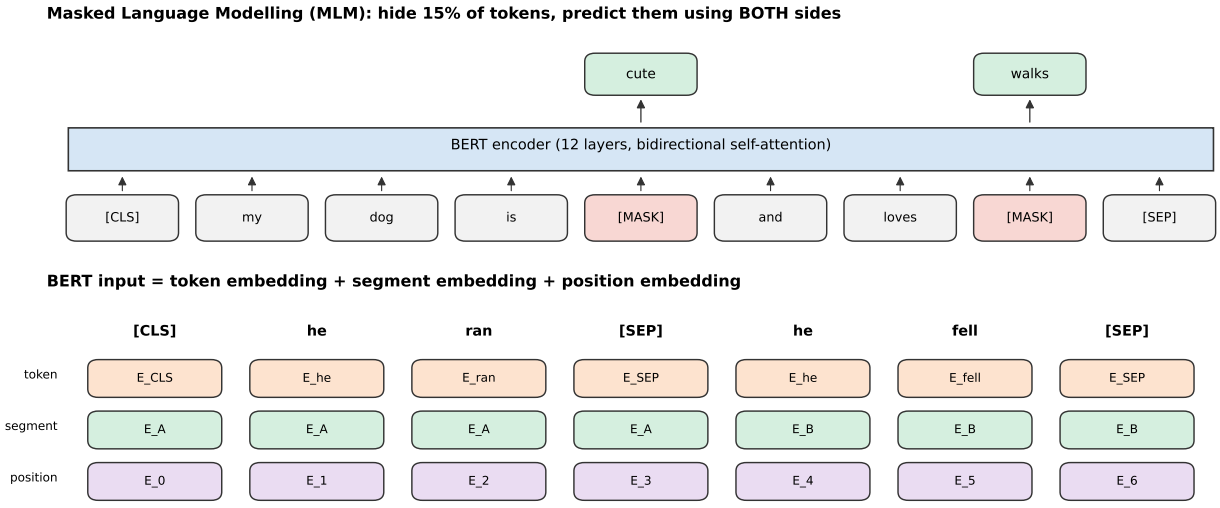


Figure 11.1: Top: masked language modelling. Bottom: how BERT builds its input vectors.

11.4 Fine-tuning BERT for tasks

Task	What to add on top
Sentence classification (sentiment, intent)	Dense + softmax on the [CLS] vector
Sentence-pair tasks (similarity, entailment)	Feed "A [SEP] B"; classify [CLS]
Token classification (NER, POS)	Dense + softmax on every token's vector
Extractive QA (SQuAD-style)	Predict the start and end token of the answer span in the passage

Typical fine-tuning settings: learning rate 2e-5 to 5e-5, batch 16–32, 2–4 epochs, AdamW with warm-up.

Hands-on in brief

BERT fine-tuning with Hugging Face. Load `AutoTokenizer` and `AutoModelForSequenceClassification.from_pretrained("bert-base-uncased", num_labels=2)`; tokenise with padding/truncation (e.g. max length 128); train with `Trainer` (or a Keras/PyTorch loop); evaluate accuracy/F1; save and serve with `pipeline("text-classification")`.

11.5 BERT variants

Model	Year	What changed
RoBERTa	2019	Drops NSP, dynamic masking, much more data, bigger batches, longer training – better accuracy
DistilBERT	2019	Knowledge distillation: about 40% smaller and 60% faster, keeping about 97% of BERT's language-understanding performance
ALBERT	2019	Shares weights across layers + factorised embeddings ⇒ far fewer parameters; sentence-order prediction instead of NSP
ELECTRA	2020	"Replaced token detection": a small generator swaps tokens, the main model spots which are fake – learns from every token, very efficient
DeBERTa	2020	Disentangled attention (content and position separately); strong on benchmarks
Domain BERTs	–	BioBERT, ClinicalBERT, FinBERT, LegalBERT – further pre-trained on domain text
Multilingual	–	mBERT (about 100 languages), XLM-RoBERTa (100 languages), IndicBERT for Indian languages
Sentence-BERT	2019	Siamese fine-tuning to produce sentence embeddings for search and similarity

11.6 Making models smaller and faster

Knowledge distillation Train a small **student** to copy the soft output probabilities of a large **teacher**: $\mathcal{L} = \alpha\,\mathcal{L}_{\text{hard labels}} + (1-\alpha)\,\mathrm{KL}(p^{(T)}_{\text{teacher}} \,\|\, p^{(T)}_{\text{student}})$, where T is a temperature that softens the probabilities.

Quantisation Store weights in fewer bits (FP32 → FP16/INT8/4-bit). INT8 is 4× smaller than FP32 and usually faster, with a small accuracy loss.

Pruning Remove weights, heads or layers that contribute little.

Parameter-efficient fine-tuning (PEFT) Freeze the big model and train small add-ons. **LoRA**: $W' = W + BA$ with low-rank $B \in \mathbb{R}^{d\times r}$, $A \in \mathbb{R}^{r\times k}$, $r \ll d$ – often well under 1% of the parameters are trained. QLoRA = LoRA on a 4-bit quantised model.

Runtime tricks ONNX Runtime/TensorRT, dynamic padding, shorter max length, batching, caching.

CHAPTER 12

Large Language Models (LLMs)

12.1 What is an LLM?

The idea in plain words

An LLM is a very large **decoder-only Transformer** trained to do one thing: **predict the next token**. Trained on trillions of tokens, this simple objective forces it to absorb grammar, facts, reasoning patterns and style. Adding instruction tuning and human feedback turns it into a helpful assistant.

Key equations

Next-token objective: $\mathcal{L} = -\sum_t \log P(x_t \mid x_1, \ldots, x_{t-1})$

Generation: predict a distribution, pick a token, append it, repeat (autoregressive).

12.1.1 How an assistant LLM is built

1. **Pre-training** – next-token prediction on huge text and code corpora (the expensive part).
2. **Supervised fine-tuning (SFT) / instruction tuning** – learn from examples of good prompt–response pairs.
3. **Alignment** – RLHF (reinforcement learning from human feedback: a reward model trained on human preference rankings guides further training) or simpler preference methods such as DPO. Makes answers more helpful, honest and safe.

12.1.2 Key vocabulary

- **Token** – a sub-word piece; in English one token is roughly 3/4 of a word on average. Pricing and limits are counted in tokens.
- **Context window** – the maximum number of tokens the model can read at once (prompt + answer).
- **Parameters** – learned weights; GPT-3 (2020) had 175 billion.
- **Prompt engineering** – clear instructions, a role, examples (few-shot), required output format, step-by-step reasoning.
- **Hallucination** – confident but false output; reduce with grounding (RAG), lower temperature and verification.

12.1.3 Decoding settings

Key equations

$P(w_i) = \dfrac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}$ $\quad T$ = temperature.

Top-k: sample only from the k most likely tokens. **Top-p (nucleus)**: sample from the smallest set of tokens whose probabilities add up to p (e.g. 0.9).

Use low temperature (0–0.3) for extraction, classification and code; higher (0.7–1.0) for brainstorming and creative writing.

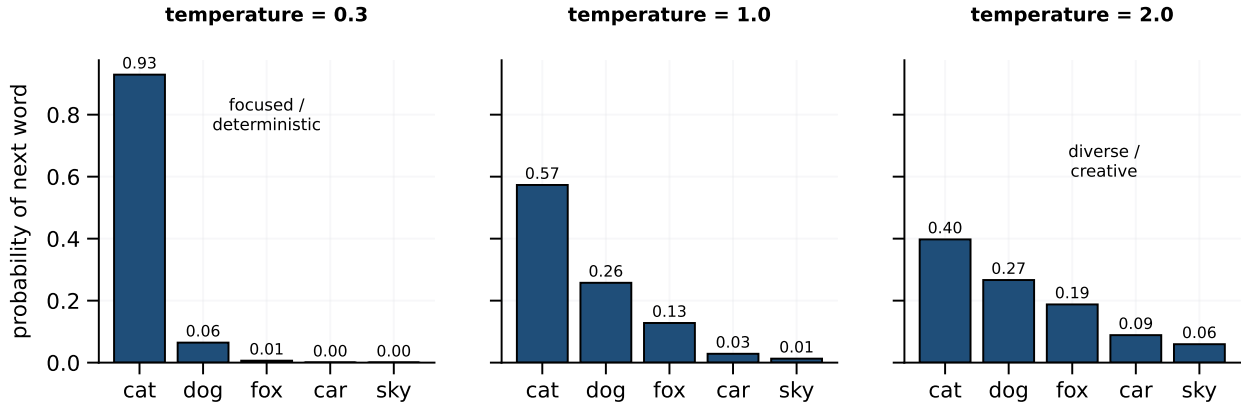


Figure 12.1: Temperature reshapes the next-token probabilities: low = predictable, high = creative but riskier.

12.2 LLM applications

12.2.1 Summarisation

- **Extractive** – copy the most important sentences (e.g. TextRank). Faithful but choppy.
- **Abstractive** – write a new summary in its own words (BART, T5, Pegasus, LLMs). Fluent, but may hallucinate.
- Long documents: chunk → summarise each → summarise the summaries (map-reduce), or use a long-context model.
- Evaluation: **ROUGE** (n-gram overlap with a reference: ROUGE-1, ROUGE-2, ROUGE-L), BERTScore, and human or LLM-as-judge checks of faithfulness.

12.2.2 Sentiment analysis

Three levels of approach: (1) lexicon/rules (VADER); (2) a fine-tuned encoder (BERT/RoBERTa) – cheap and accurate at scale; (3) zero- or few-shot prompting of an LLM – no training data needed, handles sarcasm and aspects ("food great, service slow" → food: positive, service: negative). Ask the LLM for a fixed JSON output so results are easy to parse.

12.2.3 Conversational AI

- **Chat format** – a list of messages with roles: *system* (rules, persona), *user*, *assistant*. The model is stateless: the app re-sends the conversation history each turn (trimmed or summarised to fit the context window).
- **RAG (retrieval-augmented generation)** – search your documents and paste the relevant chunks into the prompt so answers are grounded and current.
- **Tools / function calling and agents** – the model asks the app to call an API (weather, database, calculator), then uses the result; agents chain many such steps.
- **Guardrails** – input/output filtering, PII masking, refusal policies, human hand-off.

12.2.4 Multilingual capabilities

Modern LLMs are trained on many languages and can translate, answer and summarise across them, including code-mixed text (Hinglish). Quality is usually best in high-resource languages; low-resource languages and scripts may need more tokens per word (higher cost) and evaluation by native speakers. Multilingual encoders (mBERT, XLM-R, IndicBERT) and multilingual embedding models enable cross-lingual search and RAG.

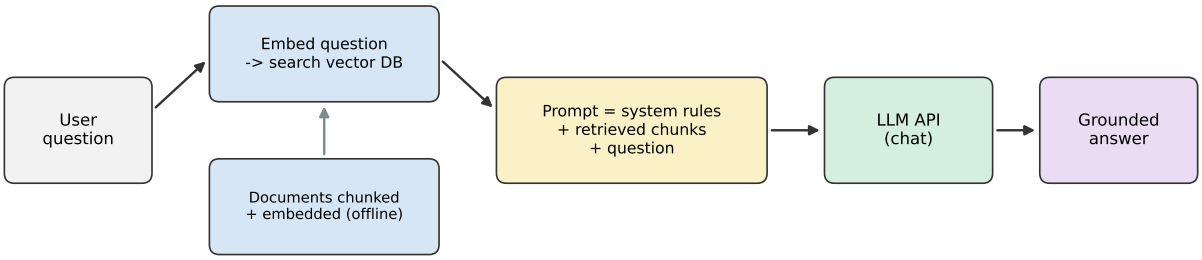


Figure 12.2: A RAG chatbot: retrieve relevant chunks, build the prompt, call the LLM.

12.3 The LLM landscape

The market changes month to month, so learn the **families and trade-offs** rather than version numbers.

Family	Provider	Notes
GPT (incl. GPT-5 series)	OpenAI	Closed weights; broad ecosystem; reasoning variants
Claude	Anthropic	Closed; strong at coding, long documents, careful writing
Gemini	Google	Closed; natively multimodal, very long context
Llama	Meta	Open weights; self-host, fine-tune, data privacy
Mistral / Mixtral	Mistral AI	Open and commercial; efficient mixture-of-experts models
DeepSeek, Qwen	DeepSeek, Alibaba	Strong open-weight models, good value for maths and code

Choosing: closed APIs = easiest, usually the strongest, but you pay per token and send data to a provider. Open-weight models = control, privacy, customisation and fixed infrastructure cost, but you run and tune them. Compare on your own task with a small evaluation set: quality, latency, cost, context length, language support and compliance.

12.4 API integration

1. Get an API key; store it in an environment variable or `.env` file – never hard-code or commit it.
2. Send a request: model name, `messages` (system + user), and parameters such as `temperature`, `max_tokens` (or the provider's equivalent), `top_p`.
3. Read the response text and the **token usage** (cost = input tokens × input price + output tokens × output price).
4. Production: retries with exponential back-off for rate limits (HTTP 429) and timeouts; streaming for faster first words; structured output (JSON schema); caching; logging and monitoring; prompt versioning; evaluation sets; content safety.
5. Orchestration frameworks (LangChain, LangGraph, LlamaIndex) help with chains, RAG, memory and agents.

Watch out

Never send confidential or personal data to an external API without approval. Treat LLM output as untrusted input: validate JSON, and check facts that matter.

CHAPTER 13

Generative AI & Diffusion Models

13.1 Introduction to generative AI

The idea in plain words

Generative AI creates new content – text, images, audio, video, code – that resembles its training data, usually steered by a natural-language prompt. Under the hood: **Transformers** for text (LLMs), **diffusion models** for images and video, often joined by models like **CLIP** that connect text and images.

Modality	Typical model type	Examples
Text, code	Decoder-only Transformer (LLM)	GPT, Claude, Gemini, Llama
Images	Diffusion (latent), some autoregressive	Stable Diffusion, DALL-E, Imagen, Midjourney
Video	Video diffusion / diffusion Transformers	Veo, Kling, Runway Gen, Stable Video Diffusion
Audio, speech	Transformers, diffusion, codec models	text-to-speech, music generation

Concerns to know: hallucination, bias, copyright and training-data consent, deepfakes (watermarking, provenance labels), privacy, and compute/energy cost.

13.2 Diffusion models: theory

The idea in plain words

Destroying an image with noise is easy; diffusion models learn to **undo** it. During training, take a real image, add a random amount of noise, and teach a network to predict that noise. To generate, start from pure random noise and repeatedly remove the predicted noise, a little at a time, until a clean image appears.

Real-life analogy

A sculptor who sees a rough block (noise) and chips away a little at each step until a statue emerges – having learned, from thousands of examples, what to remove.

13.2.1 Forward process (fixed, no learning)

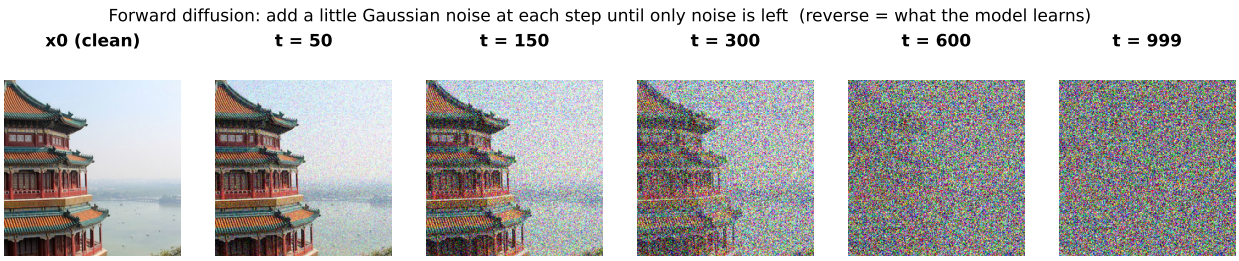


Figure 13.1: Real forward diffusion on a photo with the standard linear schedule (1,000 steps).

Key equations

One step: $q(x_t \mid x_{t-1}) = \mathcal{N}(\sqrt{1-\beta_t}\, x_{t-1},\ \beta_t I)$ (β_t = small noise amount, e.g. $10^{-4} \to 0.02$)

Jump straight to any step t: $\boxed{x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1-\bar{\alpha}_t}\, \varepsilon}$, $\varepsilon \sim \mathcal{N}(0, I)$, $\bar{\alpha}_t = \prod_{s \le t}(1-\beta_s)$

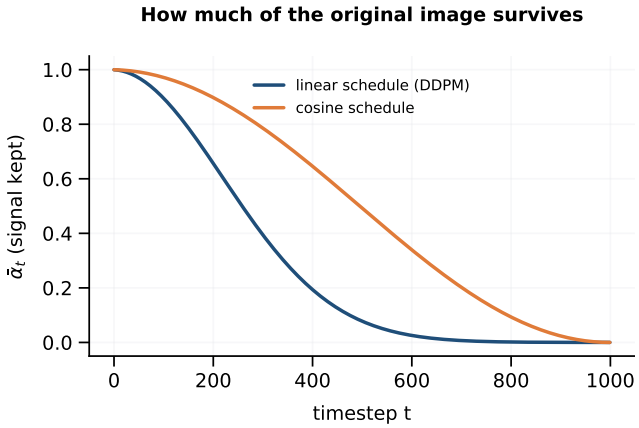


Figure 13.2: $\bar{\alpha}_t$ = share of the original signal left at step t. The cosine schedule destroys information more gradually.

13.2.2 Reverse process (learned)

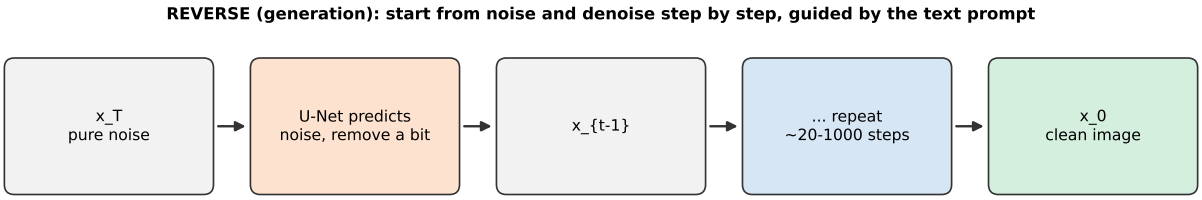


Figure 13.3: Generation runs the process backwards.

Key equations

Training loss (DDPM, 2020): $\mathcal{L} = \| \varepsilon - \varepsilon_\theta(x_t,\, t) \|^2$

Pick a random image, a random step t and random noise ε; make x_t; ask the network ε_θ to predict the noise. That's all – a simple, stable regression loss (no adversarial game).

- **Denoiser network**: usually a **U-Net** (Chapter 5) with the timestep t given as an embedding; newer models use **Diffusion Transformers (DiT)**.
- **Sampling**: DDPM uses many steps (up to 1,000). **DDIM** and other fast samplers reach good quality in about 20–50 steps.
- **Conditioning on text**: the U-Net reads text embeddings through **cross-attention** (queries from image features, keys/values from text tokens).

Key equations

Classifier-free guidance: $\hat{\varepsilon} = \varepsilon_\theta(x_t, \varnothing) + w(\varepsilon_\theta(x_t, \text{text}) - \varepsilon_\theta(x_t, \varnothing))$

w = guidance scale (often about 5–10). Higher w follows the prompt more strictly but reduces variety.

13.3 CLIP: connecting text and images

The idea in plain words

CLIP (OpenAI, 2021) trains an image encoder and a text encoder together on hundreds of millions of image–caption pairs so that matching pairs get similar vectors (contrastive learning). It gives diffusion models a way to understand prompts, and enables zero-shot image classification.

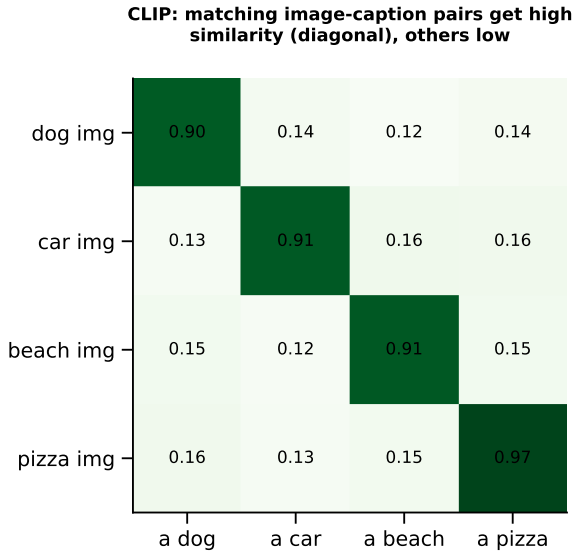


Figure 13.4: Contrastive training pushes matching image–text pairs together (diagonal) and others apart (illustrative).

13.4 Stable Diffusion

The idea in plain words

Running diffusion on full-size pixels is slow. **Latent diffusion** (Rombach et al., 2022 – the basis of Stable Diffusion) first compresses images with a **VAE** into a small latent (a 512×512×3 image becomes 64×64×4), runs diffusion there, then decodes. About 48× fewer values per image – fast enough for consumer GPUs. Open weights made it hugely popular.

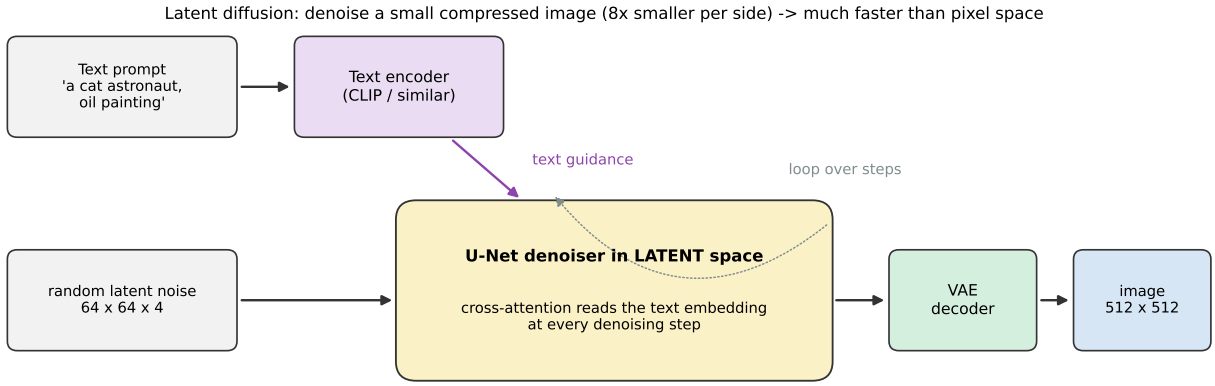


Figure 13.5: Stable Diffusion pipeline: text encoder → U-Net denoising in latent space with cross-attention → VAE decoder.

Three parts: **text encoder** (CLIP in SD 1.x; later versions add larger encoders), **U-Net denoiser** (or a Transformer in later versions), **VAE**. Settings you will meet: prompt and *negative prompt*,

number of steps, guidance scale, seed (same seed = same image), sampler, image size.

13.5 DALL-E

- **DALL-E (2021)** – a Transformer generating discrete image tokens (from a discrete VAE) after the text tokens.
- **DALL-E 2 (2022)** – "unCLIP": a prior turns the CLIP text embedding into a CLIP image embedding, then a diffusion decoder generates the image.
- **DALL-E 3 (2023)** – trained on much more detailed synthetic captions, so it follows long, complex prompts better; integrated with ChatGPT, which rewrites the user's prompt.

	Stable Diffusion	DALL-E 2/3
Access	Open weights; run locally or via APIs	Closed; via OpenAI products/API
Core	Latent diffusion + VAE + text encoder	CLIP-based prior + diffusion (DALL-E 2); improved captions (DALL-E 3)
Customisation	Very high: LoRA, DreamBooth, ControlNet, community models	Limited, prompt-based
Strength	Control, cost, privacy	Prompt following, ease of use

13.6 Image generation in practice

Text-to-image Generate from a prompt. Good prompts state subject, style, lighting, composition and camera/lens ("portrait of an old fisherman, soft window light, 85mm, film grain").

Image-to-image Start from an existing image with partial noise; *strength* controls how much it may change.

Inpainting / outpainting Regenerate only a masked area, or extend the canvas.

ControlNet Extra conditioning such as edges, depth maps or human pose to control the layout precisely.

Personalisation DreamBooth (fine-tune on 3–20 photos of a subject) and **LoRA** (small adapter weights for a style or character).

Upscaling Super-resolution models add detail at higher resolutions.

Quality is judged with FID (realism), CLIP score (prompt match) and human preference.

13.7 Video generation models

The idea in plain words

Video = a sequence of images that must also be **consistent over time**: the same character, smooth motion, believable physics. Video models extend image diffusion by adding **temporal layers** – attention across frames – and usually work in a compressed video latent space.

- **Approaches**: (1) inflate an image U-Net with temporal convolution/attention layers (e.g. Stable Video Diffusion); (2) **diffusion Transformers on spacetime patches** – the video latent is cut into small 3-D patches that act like tokens (the approach OpenAI described for Sora); (3) cascades that generate low-resolution, low-frame-rate video and then upscale in space and time.
- **Inputs**: text-to-video, image-to-video (animate a still), video-to-video (restyle or edit).
- **Landscape (late 2026)**: Google Veo, Kling (Kuaishou), Runway's Gen series, Seedance (ByteDance), Pika and Luma are widely used; some now generate synchronised audio. OpenAI

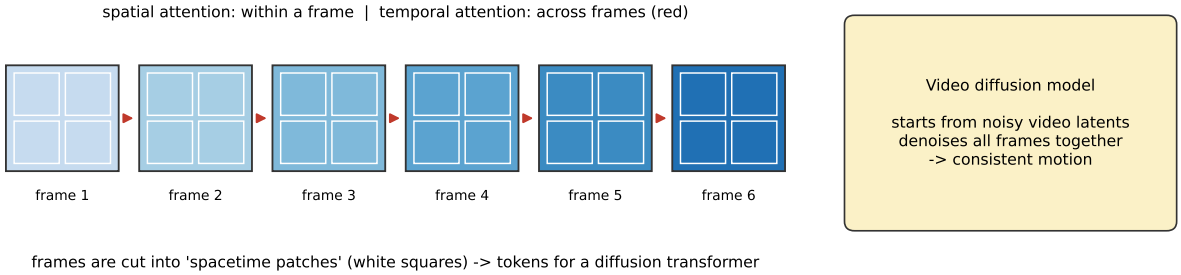


Figure 13.6: Video diffusion: spatial attention within frames, temporal attention across frames, and spacetime patches as tokens.

has reportedly discontinued its Sora app and API in 2026. The field moves fast – check current documentation.

- **Hard problems**: long-range consistency across shots, complex physics and hands, readable text in video, very high compute cost, and deepfake risks (hence watermarking and content credentials).

CHAPTER 14

Cheat Sheet & Interview Revision

14.1 Which architecture for which data?

Data / task	Architecture	Notes
Tabular	Gradient boosting first; MLP if very large	DL rarely wins on small tables
Image classification	CNN (ResNet, EfficientNet) or ViT + transfer learning	Augment; start pre-trained
Object detection	YOLO (real-time), Faster R-CNN (accuracy), SSD	Measure mAP
Segmentation	U-Net (semantic, medical), Mask R-CNN (instance)	Dice / IoU
Short sequences / time series	LSTM, GRU, 1-D CNN	Compare with a naive baseline
Text classification	Fine-tuned BERT / RoBERTa / DistilBERT	TF-IDF + LR as baseline
Translation, summarisation	Encoder–decoder Transformer (T5, BART) or LLM	ROUGE / BLEU
Chat, Q&A over documents	LLM + RAG	Ground answers, evaluate
Image generation	Latent diffusion (Stable Diffusion), DALL-E	Guidance, seeds, LoRA
Video generation	Video diffusion / diffusion Transformers	Temporal consistency

14.2 Formula sheet

Concept	Formula
Neuron	$a = g(\mathbf{w}^\top \mathbf{x} + b)$
Gradient descent	$w \leftarrow w - \eta\, \partial L/\partial w$
Conv output size	$(W - K + 2P)/S + 1$
Conv parameters	$(K \cdot K \cdot C_{\text{in}} + 1) \cdot C_{\text{out}}$
Batch norm	$y = \gamma \frac{x - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}} + \beta$
Residual block	$y = F(x) + x$
IoU / Dice	$\frac{\lvert A \cap B \rvert}{\lvert A \cup B \rvert}$ / $\frac{2\lvert A \cap B \rvert}{\lvert A \rvert + \lvert B \rvert}$
RNN	$h_t = \tanh(W_{xh} x_t + W_{hh} h_{t-1} + b)$
LSTM cell	$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t, \quad h_t = o_t \odot \tanh(c_t)$
GRU	$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$
Attention (seq2seq)	$c_t = \sum_i \alpha_{t,i} h_i, \quad \alpha = \text{softmax}(e)$
Scaled dot-product attention	$\text{softmax}(QK^\top/\sqrt{d_k})\, V$
GAN	$\min_G \max_D \ \mathbb{E}[\log D(x)] + \mathbb{E}[\log(1 - D(G(z)))]$
VAE	$z = \mu + \sigma \odot \varepsilon$; loss = reconstruction + KL
Diffusion	$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t}\, \varepsilon$; $\quad$ loss $\lVert \varepsilon - \varepsilon_\theta(x_t, t) \rVert^2$
Temperature	$\text{softmax}(z/T)$

14.3 Interview quick revision

Why do we need activation functions? Without them, any stack of layers is just one linear function.

Explain backpropagation. Chain rule applied from the loss backwards to get each weight's gradient; then gradient descent updates the weights.

Why ReLU over sigmoid in hidden layers? Sigmoid's gradient is at most 0.25 and vanishes in deep nets; ReLU's is 1 for positive inputs and cheap to compute.

What does dropout do? Randomly turns off neurons during training so the network can't overrely on any of them – a regulariser.

Why batch normalisation? Stabilises layer inputs, allowing faster training and higher learning rates.

Why CNNs for images? Local connectivity and weight sharing: far fewer parameters and translation invariance.

What does pooling do? Downsamples feature maps, reducing computation and adding tolerance to small shifts. No parameters.

Why did VGG use only 3×3 filters? Stacks of 3×3 give the same receptive field as larger filters with fewer weights and more non-linearity.

What problem does ResNet solve? Degradation/vanishing gradients in very deep networks – skip connections ($F(x) + x$) let gradients flow.

What is transfer learning? Reuse a model pre-trained on a large dataset; freeze it, train a new head, then optionally fine-tune with a tiny learning rate.

One-stage vs two-stage detectors? Two-stage (Faster R-CNN) propose regions then classify – accurate. One-stage (YOLO, SSD) predict in one pass – fast.

What is NMS? Keeps the highest-scoring box and removes overlapping boxes above an IoU

threshold.

Semantic vs instance segmentation? Semantic labels every pixel by class; instance also separates individual objects.

Why do RNNs suffer from vanishing gradients? The gradient is a product of many per-step factors; below 1 it shrinks exponentially.

How does LSTM fix it? An additive cell state controlled by gates gives gradients a near-constant path back through time.

LSTM vs GRU? LSTM: 3 gates + separate cell state. GRU: 2 gates, no cell state, fewer parameters, often similar accuracy.

What is attention? A learned, weighted average over all input states, with weights given by relevance to the current step.

Why divide by $\sqrt{d_k}$? Large dot products saturate the softmax and shrink gradients; scaling keeps them in a good range.

Why positional encoding? Self-attention ignores order; position vectors add it back.

BERT vs GPT? BERT: encoder-only, bidirectional, masked-token pre-training – understanding tasks. GPT: decoder-only, left-to-right next-token prediction – generation.

What is MLM? Mask 15% of tokens and predict them from both sides of the context.

GAN vs VAE vs diffusion? GAN: adversarial, sharp, unstable, mode collapse. VAE: stable, smooth latent, blurry. Diffusion: stable, high quality and diversity, slower sampling.

What is mode collapse? The generator produces only a few kinds of outputs that fool the discriminator.

Why is Stable Diffusion fast? It denoises in a small VAE latent space (8× smaller per side) instead of full pixels.

What does the guidance scale do? Controls how strongly generation follows the prompt (classifier-free guidance).

What is RAG? Retrieve relevant documents and add them to the prompt so the LLM answers from real, current sources.

What does temperature do? Divides logits before softmax: low = deterministic, high = diverse.

Further Reading

1. I. Goodfellow, Y. Bengio, A. Courville. *Deep Learning*. MIT Press, 2016.
2. F. Chollet. *Deep Learning with Python*, 2nd ed. Manning, 2021 (Keras).
3. A. Zhang et al. *Dive into Deep Learning* (d2l.ai) – free, interactive.
4. Original papers: LeCun et al. 1998 (LeNet); Krizhevsky et al. 2012 (AlexNet); Simonyan & Zisserman 2014 (VGG); He et al. 2015 (ResNet); Ren et al. 2015 (Faster R-CNN); Redmon et al. 2016 (YOLO); Ronneberger et al. 2015 (U-Net); He et al. 2017 (Mask R-CNN); Hochreiter & Schmidhuber 1997 (LSTM); Cho et al. 2014 (GRU); Bahdanau et al. 2014 (attention); Vaswani et al. 2017 (Transformer); Devlin et al. 2018 (BERT); Goodfellow et al. 2014 (GAN); Kingma & Welling 2013 (VAE); Ho et al. 2020 (DDPM); Rombach et al. 2022 (latent diffusion); Radford et al. 2021 (CLIP).
5. Keras, TensorFlow, PyTorch and Hugging Face documentation.